https://pubsonline.informs.org/journal/mksc

MARKETING SCIENCE
Vol. 45, No. 2, March–April 2026, pp. 258–279
ISSN 0732-2399 (print), ISSN 1526-548X (online)

A Deep-DiD Method to Estimate Heterogeneous Treatment Effects: Application to Content Creator Selection

Yan Cheng,[a,*] Jingbo Wang,[b,*] Xinyu Cao,[b,*] Zuo-Jun Max Shen,[c] Yuhui Zhang[d]

[a] College of Business, Shanghai University of Finance and Economics, Shanghai 200433, China; [b] Department of Marketing, The Chinese University of Hong Kong, Hong Kong SAR, China; [c] Department of Industrial Engineering and Operations Research & Department of Civil and Environmental Engineering, The University of Hong Kong, Hong Kong SAR, China; [d] Industry Collaborator

*Corresponding authors

Contact: chengyan@mail.shufe.edu.cn, https://orcid.org/0000-0002-7625-5857 (YC); jingbowang@cuhk.edu.hk, https://orcid.org/0000-0001-9612-4558 (JW); xinyucao@cuhk.edu.hk, https://orcid.org/0000-0001-8309-9947 (XC); maxshen@hku.hk (Z-JMS); yuhuizhanglink@outlook.com (YZ)

Received: November 1, 2023
Revised: October 28, 2024; June 16, 2025
Accepted: August 4, 2025
Published Online in Articles in Advance: January 14, 2026

https://doi.org/10.1287/mksc.2023.0511

Copyright: © 2026 INFORMS

Abstract. In this paper, we propose a Deep-DiD method that incorporates two deep neural networks in a difference-in-difference (DiD) framework to estimate heterogeneous treatment effects (HTEs). The dual-network architecture contains one neural network modeling HTEs as a nonparametric function of pretreatment features and another neural network capturing individual and time fixed effects. Through a series of simulations, we show that our method can uncover the true HTEs with high accuracy under various settings and demonstrates more robust estimation performance compared with existing methods like linear models and random forests. We apply this method to an empirical setting where a large video-sharing platform introduced a "Creator Signing Program" aimed at signing creators and motivating them to generate more high-quality video content. Leveraging a matched data set of signed and unsigned creators, we employ our Deep-DiD method to estimate the HTEs of the signing program. Our method can help the platform optimize creator selection by identifying creators with the highest-estimated treatment effects. Through out-of-sample tests, we show that creators selected by the Deep-DiD method experience substantially larger actual performance jumps than those selected by the platform. Creator selection based on the Deep-DiD method also consistently outperforms that based on linear models.

History: Olivier Toubia served as the senior editor. This paper has been accepted for the *Marketing Science* Special Section on Digital Platforms in Marketing Science.

Funding: This research was supported by Hong Kong Research Grants Council [Project No. 14504823] and Shanghai Research Center for Data Science and Decision Technology.

Supplemental Material: The online appendix and data files are available at https://doi.org/10.1287/mksc.2023.0511.

Keywords: content-sharing platform • creator economy • deep neural networks • Difference-in-Difference • heterogeneous treatment effects

1. Introduction

Difference-in-Difference (DiD) is a commonly used method to estimate the average treatment effect on the treated in a quasi-experimental setting, with the key identifying assumption that the average outcome in the treated and control groups would have followed parallel trends in the absence of treatment (Roth et al. 2023). However, this estimate only allows us to understand whether and by how much an intervention is effective on average. The explosion of data now makes it possible to explore how treatment effects vary across individuals with different characteristics. By understanding how individuals respond to interventions differently, companies can customize their interventions to maximize effectiveness. Thus, estimating heterogeneous treatment effects (HTEs) can significantly enhance various marketing practices, such as pricing strategies, product recommendations, and marketing campaign optimization (see, e.g., Fong et al. 2015, Ascarza 2018, and Hitsch et al. 2024).

Although the availability of rich data allows for high-dimensional pretreatment features, estimating HTEs parametrically becomes more challenging because there can be complex interactions between pretreatment features, and it is difficult to assume a correct functional relationship between HTEs and pretreatment features. Existing works have proposed several nonparametric approaches for the estimation of HTEs, including Bayesian nonparametric methods (see, e.g., Hill 2011) and tree-based methods (see, e.g., Wager and Athey 2018).

Farrell et al. (2021) demonstrated the use of DNN for estimating causal parameters, and Farrell et al. (2020) presented a DNN-based framework that can

estimate fully flexible nonparametric individual heterogeneity. Inspired by these works, we integrate deep neural networks (DNN) in a DiD framework to estimate HTEs. We propose a dual-network architecture consisting of two DNN structures. One DNN structure captures the nonparametric relationship between the treatment effect $\beta(\cdot)$ and pretreatment features $\mathbf{W}_i$. The other DNN structure summarizes high-dimensional individual fixed effects μ_i and time fixed effects τ_t into a nuisance function $f(\mu_i, \tau_t)$, which also allows for possible interactions between them. The combination of these two parts forms a semiparametric DiD model with DNNs embedded. We name it the *Deep-DiD* model.

Through simulations, we demonstrate that our Deep-DiD method can uncover the true HTEs with high accuracy across various functional forms of the true $\beta(\cdot)$ and $f(\cdot,\cdot)$. We also show that the embedded DNN structures demonstrate more robust estimation performance than linear estimators and random forests. We further examine how the estimation performance of our approach varies with sample sizes, feature dimensions, and different implementation choices. Overall, our Deep-DiD method demonstrates high accuracy and robustness in estimation.

We apply our method to an empirical setting in which a large video-sharing platform launched a "Creator Signing Program" ("the signing program" hereafter) aimed at identifying promising creators and encouraging them to produce more high-quality content. The program was initially launched in three countries located in South America, Africa, and Europe.[1] Signed creators received monthly payments based on their performance. We apply our method to estimate the HTEs of the signing program and further demonstrate how our estimation can aid the platform in optimizing creator selection.

We construct a matched data set with 2,343 creators who were signed from May 1–June 30, 2022 (the treated group) and 2,343 creators who remained unsigned (the control group). For each creator, we observed their activities in the 89 days before and 90 days after the signing date.[2] We define two performance measures: the total user viewing time contributed by the creator's videos and the total user engagement contributed by the creators' videos. The two groups show parallel trends in daily average performance measures before treatment. Using a canonical DiD model, we show that the signing program substantially boosts the performance metrics of signed creators.

We further prepare the data for estimating the Deep-DiD model and testing its performance in creator selection. We construct a set of variables ($\mathbf{W}_i$) that describe creators' pretreatment characteristics in three aspects: Creator Production and Viewer Responses, Visual Features, and Audio Features. Because of the loss of some video data, the data set with the full set of pretreatment features contains 2,204 creators, with 1,126 from the treated group and 1,078 from the control group. We randomly split this data set into two parts: 60% (1,322 creators) for model training and validation and 40% (882 creators) for out-of-sample testing. The trained model provides an estimated function $\hat{\beta}(\cdot)$, which we can apply to calculate $\hat{\beta}_i = \hat{\beta}(\mathbf{W}_i)$ for each i in both the training-validation and test sets. The estimated treatment effects reveal substantial heterogeneity across creators.

Our method can help the platform optimize creator selection. By utilizing our estimated function $\beta(\mathbf{W}_i)$ and each creator's pretreatment features $\mathbf{W}_i$, the platform can estimate how much the signing program will change the performance of each creator. The platform can then select the creators with the highest estimated treatment effects to sign.

We evaluate this creator selection method through out-of-sample tests. Among the 882 creators in the test set, 441 were signed by the platform (labeled "Platform Selected"). We also select 441 creators with the highest estimated treatment effects (labeled "Model Selected") and compare them with the Platform Selected ones. The Model Selected creators show significantly higher estimated treatment effects compared with the Platform Selected creators. Furthermore, among creators signed by the platform, those selected by the model demonstrate substantially higher actual performance jumps compared with those not selected by the model. Here, a creator's performance jump is defined as his or her daily average performance after the signing date minus that before the signing date. This measure can be considered an "empirically observed" treatment effect of the signing program on signed creators, although it is not strictly causal. These two evaluation results collectively demonstrate that creator selection based on our Deep-DiD method outperforms the platform's selection. We also show that creator selection based on our Deep-DiD method consistently performs better than selection based on linear models.

1.1. Contributions to Literature

Methodologically, our paper contributes to the literature on machine-learning-based causal inference. Machine-learning-based causal inference methods are advancing rapidly, driven by the availability of big data and powerful computing platforms. These advancements enable researchers and practitioners to examine new empirical scenarios that go beyond the capabilities of traditional methods. For instance, they allow for the handling of high-dimensional covariates (Belloni et al. 2017) and the estimation of heterogeneous treatment effects for optimal targeting (Hitsch et al. 2024). Wager and Athey (2018) proposed to estimate heterogeneous treatment effects using

random forests. Applying this method, Guo et al. (2021) estimated the heterogeneous treatment effects of the Physician Payment Sunshine Act disclosure on pharmaceutical payments to doctors, whereas Zhang and Luo (2023) estimated the relationship between photos posted by consumers and restaurant survival. Cong et al. (2021) employed the orthogonal random forest method proposed by Oprescu et al. (2019) to estimate the heterogeneous treatment effect of livestreaming.

As a significant recent development in this field, Farrell et al. (2021) demonstrated the use of DNNs for estimating causal parameters. They further extended it to estimate heterogeneous treatment effects Farrell et al. (2020). To the best of our knowledge, our paper is the first empirical study to implement the DNN-based heterogeneity architecture proposed in Farrell et al. (2020). We innovatively propose a dual-network structure that combines two DNNs in a DiD framework, enabling estimation of HTEs with both high-dimensional pretreatment features and high-dimensional fixed effects. Through simulations and an empirical application, we demonstrate the robust and superior performance of this DNN-based approach.

Our paper also contributes to the literature on modern variations of DiD methods that deal with heterogeneous treatment effects. Two-way fixed-effects (TWFE) estimator is a DiD estimator that accounts for both unit-specific and time-specific unobserved heterogeneity in panel data by including fixed effects for units (e.g., individuals, firms, states) and time periods. Chaisemartin and D'Haultfœuille (2020) examined the properties of TWFE estimators, demonstrating that the TWFE estimators can produce misleading estimates when treatment effects vary across units or over time. This is because TWFE estimators calculate a weighted sum of treatment effects, where some weights can be negative when the actual treatment effects vary across units or over time. There are several methodological works trying to deal with the bias in TWFE when HTE exists. Callaway and Sant'Anna (2021) proposed a group-time ATT (average treatment effect on the treated) estimator that computes the average treatment effect for each cohort and time period and then aggregates them using inverse probability weighting or outcome regression. Sun and Abraham (2021) fixed TWFE's biases by first estimating cohort-specific dynamic effects and then combined these effects to produce ATT. Borusyak et al. (2024) proposed an imputation-based estimator; it predicted counterfactual outcomes for treated units by estimating a linear factor model using untreated units only and then computed treatment effects by comparing the observed and predicted outcomes of treated units. All of these methods required parallel trends (i.e., in the absence of treatment, the average outcome of the treated and control groups would have followed the same trajectory over time) and no anticipatory effects (i.e., treatment effects do not occur before the actual treatment date) as key identification assumptions. Abadie and Gardeazabal (2003) and Abadie et al. (2010) proposed a synthetic control method that estimated the treatment effect on a single (or a few) treated unit(s) by constructing a weighted combination of untreated units that closely mimicked the treated unit's pretreatment outcomes and characteristics. Note that although the synthetic control method can deal with violation of the parallel-trend assumption, it is restricted to settings with a single or small number of treated units, and it is not suited when treated units' outcomes are outside the range of those for control units (Arkhangelsky et al. 2021, Li et al. 2024). Arkhangelsky et al. (2021) hybridized DiD and synthetic control method so that it could handle settings with many treated units or when treated and control units were very different. It also solved the biases in TWFE estimators when treatment effects varied across units or over time. Table 1 summarizes the papers mentioned above.

Table 1. Representative Modern DiD Variations

Paper (method)	Key issues addressed	Data requirements	Estimation outcomes
Callaway and Sant'Anna (2021)	Bias in TWFE when there exists HTE	Parallel trends No anticipation	Group-time ATT
Sun and Abraham (2021)			Group-time ATT
Borusyak et al. (2024)			Individual-level HTE
This paper	Neural network based, high-dimensional covariates		Individual-level HTE
Synthetic control method: Abadie and Gardeazabal (2003), Abadie et al. (2010)	Violation of parallel trends	A single (or a few) treated units; treated and control units' (pretreatment) outcomes not very different	Unit-specific ATT
Synthetic DiD method: Arkhangelsky et al. (2021)		Multiple treated and control units	ATT; HTE via interactions or stratifications

Notes. There are many works about modern DiD variations. We summarize only some representative modern DiD variations that are related to heterogeneous treatment effects.

Our paper is most closely related to Borusyak et al. (2024) because both approaches can directly estimate individual-level HTEs. Our DNN-based method can be understood, at a high level, as a generalization of the imputation-based approach proposed by Borusyak et al. (2024). Unlike Borusyak et al. (2024), which imputed untreated counterfactuals for treated units by estimating a linear factor model, our method uses DNN to nonparametrically impute untreated counterfactuals for treated units. DNN not only allows for more flexible imputations but also enables our method to handle high-dimensional covariates.

This paper is broadly related to the literature on the application of deep learning methods in marketing problems. For example, Liu et al. (2019) employed deep learning-based natural language processing to analyze how consumers utilize review content; Wei and Jiang (2022) demonstrated that deep neural networks can accurately estimate structural parameters with low estimation costs; Cao et al. (2025) utilized a deep learning approach to examine the impact of image-text coupling on consumer preference; Fong et al. (2022) developed an explainable deep learning classifier to predict emotional responses to music. Several studies have utilized deep learning approaches to understand the influence of images (see, e.g., Li and Xie 2020, Liu et al. 2020, Dew et al. 2022, Zhang et al. 2022, Dzyabura et al. 2023, and Zhang and Luo 2023) and videos (see, e.g., Li et al. 2019, Zhou et al. 2021, Cheng and Zhang 2024, and Yang et al. 2025) on consumer perceptions, preferences, and purchase decisions. Liu (2023) provided a comprehensive review in this area. Again, as far as we know, our paper represents the first empirical application of deep learning in estimating heterogeneous treatment effects. Furthermore, we incorporate visual features and audio features, which are derived from unstructured video data using deep-learning-based methods, as inputs in our Deep-DiD model.

Substantively, this paper contributes to the literature on content-sharing platforms, particularly regarding content creation. Existing works have empirically studied how to motivate more content creation using financial incentives and behavioral nudges (Burtch et al. 2018, Huang et al. 2019, Zeng et al. 2022, Lu et al. 2023) and have theoretically examined how platforms should compensate creators to motivate content creations (Jain and Qian 2021, Bhargava 2022). To the best of our knowledge, our paper is the first to quantify the causal impact of signing creators on their performances. Furthermore, we propose an approach based on deep neural networks that allows platforms to optimally select creators based on their comprehensive characteristics.

This paper is also related to the literature on influencer marketing and the creator economy. With the rapid growth of influencer marketing in recent years,[3] there has been a surge in research studying various aspects of influencer marketing. This includes investigations into the impact of brand sponsorship on influencers' reputation (Cheng and Zhang 2024), how influencers adjust their voices in sponsored videos (Hwang et al. 2021), the effect of live-streaming on product demand and seller survival (Cong et al. 2021, Liu et al. 2022, Huang and Morozov 2025), and the relationship between audience size and live-streaming revenues (Lu et al. 2021). Several studies have also examined influencer selection. Yang et al. (2025) proposed a motion-related metric that predicts the effectiveness of influencer ads on product sales, whereas Rajaram and Manchanda (2021) used an interpretable deep learning framework to predict the popularity of influencers' content. Leung et al. (2022) explored the factors that determine the effectiveness of influencer marketing, whereas Wies et al. (2023) and Tian et al. (2024) proposed influencer selection methods based on the number of followers and follower elasticity. Feng et al. (2025) predicted the potential of influencers using their facial features. It is worth noting that these studies primarily consider the influencer selection problem from the perspective of sponsoring companies, proposing metrics for selecting influencers. In contrast, the empirical application in this paper focuses on the creator selection problem from the platform's perspective; how should a platform select creators to sign and provide financial incentives? We utilize data from a real signing program to estimate the causal effects and propose a creator selection method based on the estimated HTEs. When estimating the HTEs, we make use of comprehensive characteristics of creators rather than relying on a few metrics.

The paper is structured as follows. Section 2 introduces the Deep-DiD method and shows its performance through simulations. Section 3 provides an introduction to the empirical context. In Section 4, we apply our Deep-DiD method to estimate the heterogeneous treatment effects of the signing program. Section 5 illustrates how to conduct creator selection using our model estimation results and evaluates the performance of model-based creator selection. Section 6 offers concluding remarks for the paper.

2. Deep-DiD Method

Difference-in-Difference (DiD) is a widely used method for estimating causal effects in quasi-experimental settings. The idea is to compare the difference in outcomes after and before the treatment for the treated group to that of the untreated group. The key identifying assumption is that the treated and untreated groups exhibit parallel trends in the outcome measure before treatment, and the parallel trajectory would have continued in the absence of treatment. A canonical DiD setup is

$$Y_{it} = \beta D_i \times A_t + \mu_i + \tau_t + \epsilon_{it}, \tag{1}$$

where Y_{it} denotes the outcome of interest for individual i at time t ($i = 1, \ldots, N, t = 1, \ldots, T$). The binary variable D_i indicates whether individual i belongs to the treated group ($D_i = 1$) or the control group ($D_i = 0$). The binary variable A_t denotes whether the treatment has been launched by time t. μ_i and τ_t capture individual and time fixed effects, respectively. ϵ_{it} is the error term. The interaction term's coefficient β captures the average treatment effect on the treated (ATT) (Bertrand et al. 2004).

Nowadays, more and more marketing applications require the estimation of heterogeneous treatment effects, that is, to predict how each individual would react to the treatment and how the treatment effect varies with their pretreatment features. In the DiD framework, it is essential to estimate $\beta(\cdot)$ as a function of $\mathbf{W}_i$ in the following equation:

$$Y_{it} = \beta(\mathbf{W}_i) D_i \times A_t + \mu_i + \tau_t + \epsilon_{it}, \tag{2}$$

where $\mathbf{W}_i$ is a J-dimensional vector denoting individual i's pretreatment features. $\beta(\mathbf{W}_i)$ captures the average treatment effect conditional on the pretreatment features.

An earlier practice of estimating heterogeneous treatment effect is to assume a functional form of $\beta(\mathbf{W}_i)$ and estimate it accordingly. For example, there are existing works, including three-way interactions $D_i \times A_t \times \mathbf{W}_i$ in the DiD model, to examine treatment effect heterogeneity (see, e.g., Duflo 2003), which is equivalent to assuming $\beta(\mathbf{W}_i)$ as a linear function of $\mathbf{W}_i$. However, there can be complex interactions between these pretreatment features. A linear functional form of $\beta(\mathbf{W}_i)$ cannot take into account the possible complex interactions between pretreatment features, and it is hard for researchers to know the correct specification ex ante. Therefore, this method may give a directional sense of how the treatment effect changes with a certain variable, but it may lead to biased estimates if the goal is to obtain an accurate estimate of individual-level HTEs.

Nonparametric approaches to estimation of heterogeneous treatment effects include nearest-neighbor matching, kernel methods, and series estimation. As the dimension of pretreatment features gets large, the performance of these methods deteriorates (Wager and Athey 2018), and it calls for the utilization of machine-learning methods. Wager and Athey (2018) and Athey et al. (2019) proposed to estimate heterogeneous treatment effects using random forests.

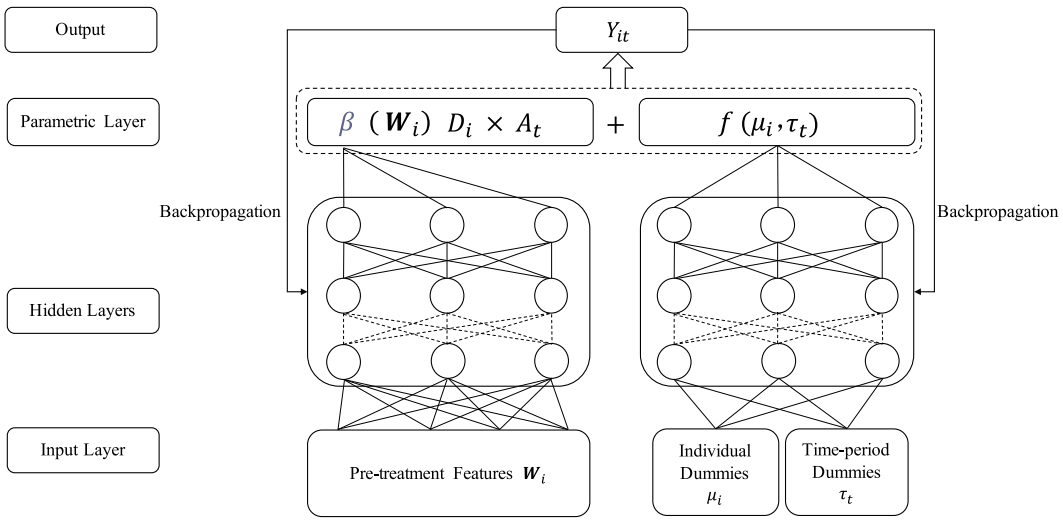
Farrell et al. (2021) showed that deep neural networks (DNN) can be used in semiparametric inference. More specifically, they showed that DNN can be used to estimate the nonparametric part (they call it a "nuisance function") in the first-step estimation with a fast enough convergence rate, which enables a valid second-step inference[4] of the parametric part. They showed the application of this method on estimating average causal parameters. Farrell et al. (2020) further presented a DNN-based framework that could estimate fully flexible nonparametric individual heterogeneity.

Inspired by this line of works, we innovatively integrate DNN with the DiD model to estimate $\beta(\mathbf{W}_i)$. We call it the *Deep-DiD* approach. We consider a more general setup, where the individual fixed effects and time fixed effects may interact and are represented by a general functional form $f(\mu_i, \tau_t)$:

$$Y_{it} = \beta(\mathbf{W}_i) D_i \times A_t + f(\mu_i, \tau_t) + \epsilon_{it}. \tag{3}$$

Figure 1 illustrates the conceptual framework of our approach. We use a dual-network architecture in estimation. One is the HTE estimation network, with $\mathbf{W}_i$ as the input layer and β_i as the output layer. This DNN characterizes β_i as a nonparametric function of $\mathbf{W}_i$. The other is the fixed-effect network, with the

Figure 1. Deep-DiD Model Architecture



high-dimensional individual dummies and time-period dummies as the input layer and a nuisance function $f(\mu_i, \tau_t)$ as the parametric layer.[5] As we will show in Section 2.1.2, this design allows for possible complex interactions between individual fixed effects and time fixed effects.

The estimation process is an iterative process. In each iteration, the mean squared loss between the outcome variable y_{it} and the sum of $\beta(\mathbf{W}_i)D_i \times A_t$ and $f(\mu_i, \tau_t)$ are computed, and then the parameters in each layer are updated to minimize this loss in the next iteration until convergence is achieved. The heterogeneous treatment effects $\beta_i = \beta(\mathbf{W}_i)$'s are retrieved from the parametric layer of the HTE estimation network.

2.1. Simulation

We conduct a series of simulations to demonstrate the performance of our Deep-DiD method. In particular, in a DiD setup with heterogeneous treatment effects as described by Equation (3), our Deep-DiD method can recover the true distribution of $\beta(\mathbf{W}_i)$ with high enough accuracy.

2.1.1. Simulation Procedure. Let us first describe the data-generating process. We generate $(\mathbf{W}_i, \mu_i, \tau_t, D_i, A_t)$ $(i = 1, \ldots, N, t = 1, \ldots, T)$, where $\mathbf{W}_i$ is a J-dimensional vector, according to certain distribution assumptions. Unless otherwise specified, we assume that each element in $\mathbf{W}_i$ is independent and follows a normal distribution $N(5,3)$. Individual fixed effects μ_i are all independent and follow a normal distribution $N(0,5)$ $(i = 1, \ldots, N)$. Time fixed effects τ_t are also independent and follow $N(0,1)$ $(t = 1, \ldots, T)$. We consider a setting that the treatment assignment is based on observables rather than random, which makes the estimation task more challenging. In particular, we assume that each individual i is treated with probability $\Pr(D_i = 1) = (1 + \exp(-\sum_j (W_{ij} - 5)/J))^{-1}$. We assume that the treatment takes place at $T/2$, that is, $A_t = 1$ when $t \geq T/2$, and $A_t = 0$ otherwise. The error terms ϵ_{it} are independent and follow $N(0,1)$.[6] Then, we compute $\beta_i = \beta(\mathbf{W}_i)$ and $f(\mu_i, \tau_t)$ according to the assumed functional forms and compute Y_{it} according to Equation (3). $(\mathbf{W}_i, Y_{it}, D_i, A_t)$ $(i = 1, \ldots, N, t = 1, \ldots, T)$ constitutes a panel data for model training. (Note that μ_i and τ_t are unobserved.) Following the same procedure, we generate another data set with the same size. This is a test data set, which is independent with the training data set and will be used to evaluate the estimation performance.

For the training data set, we further randomly split it into 80% for model training and 20% for validation. We use the training set to train the model and use MSE loss on the validation set to monitor the training progress to prevent overfitting. In both networks, we design two hidden layers. Unless otherwise specified, we set each hidden layer with 32 neurons. Rectified linear unit (ReLU) activation functions are used between layers, which is a common and effective choice for neural network architectures. We set the batch size as 512 and the number of epochs as 10. We use the Adaptive Moment Estimation (Adam) optimizer. The learning rate is initialized at 0.001 and halved if the model's performance fails to improve for two consecutive evaluation periods.[7]

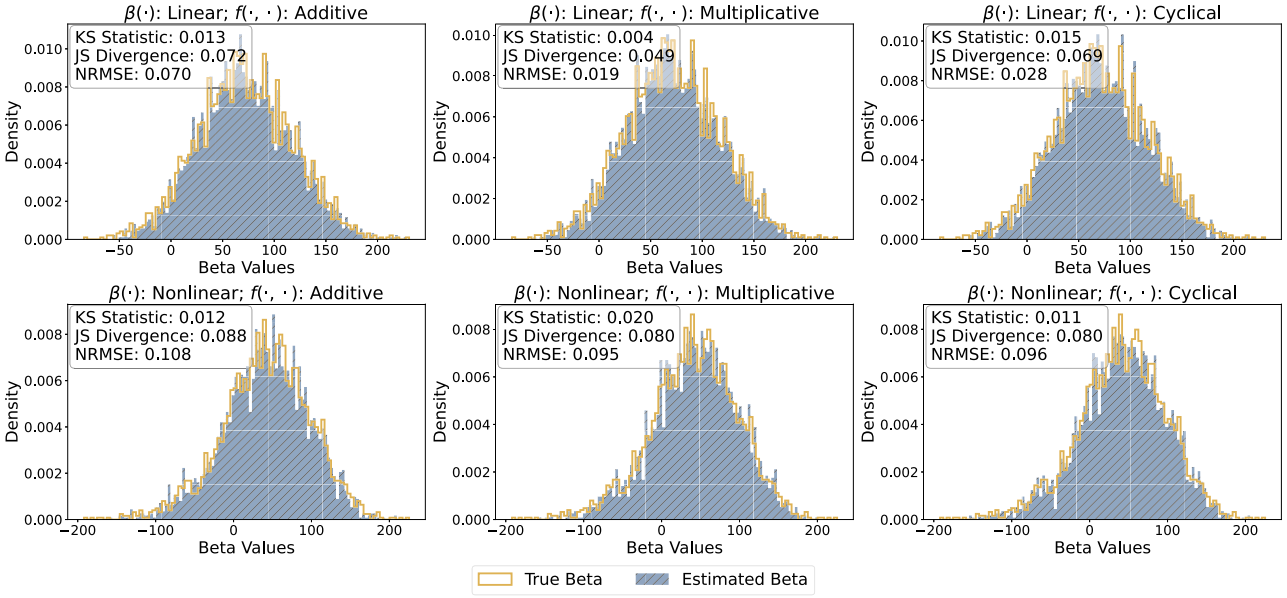
The trained model essentially provides us with an estimated (nonparametric) function $\hat{\beta}(\cdot)$. We apply it to the test set to calculate $\hat{\beta}_i = \hat{\beta}(\mathbf{W}_i)$ for each i in the test set. To evaluate the estimation performance, we compare the distribution of $\hat{\beta}_i$s to the distribution of true β_is of the test sample. We employ two commonly used metrics to quantify the difference between the two distributions: the Kolmogorov-Smirnov (KS) Statistic and the Jensen-Shannon (JS) Divergence. The KS Statistic, ranging from 0 to 1, represents the maximum vertical distance between the two cumulative distribution functions (CDFs) and is widely used in statistical analysis. The JS Divergence, also ranging from 0 to 1 when implemented in Python Scipy, is a symmetrized and smoothed version of the Kullback-Leibler divergence. For both metrics, lower values indicate greater similarity between the two distributions.[8] To further evaluate whether our method accurately predicts the value of each β_i (beyond distribution matching), we also use a third metric: Normalized Root Mean Square Error (NRMSE)= $\sqrt{\sum_i (\hat{\beta}_i - \beta_i)^2 / N} / \sigma_\beta$, which is the root of mean square error divided by the standard deviation. Whereas both mean squared error (MSE) and root mean squared error (RMSE) depend on the scale of the variable of interest, making it hard to evaluate and compare the estimation performance across different tasks, NRMSE is scale independent and comparable across different data sets with different scales. It is usually considered an excellent fit when NRMSE < 0.1.

All simulations are conducted on a local Windows 11 desktop with an Intel Core i9-14900KF processor and an NVIDIA GeForce RTX 4090 graphics card (24GB VRAM). The Python environment comprised CUDA 11.8, PyTorch 2.3.1, and Python 3.10.14.

2.1.2. Simulation Results.

Performance of Deep-DiD Under Various Settings. First, we show that under different assumptions for the true functional forms of $\beta(\cdot)$ and $f(\cdot,\cdot)$, the true β_is can be well uncovered using our Deep-DiD method (Figure 2). We try two different assumptions for the true functional forms of $\beta(\cdot)$ (linear, nonlinear) and three for $f(\cdot,\cdot)$ (additive, multiplicative, and cyclical).

Figure 2. (Color online) Estimation Performance of Deep-DiD Across Different Functional Forms of $\beta(\cdot)$ and $f(\cdot,\cdot)$



Notes. Two functional forms of $\beta(\mathbf{W}_i)$: linear $(15W_{i1} - 2W_{i2} + W_{i3})$ and nonlinear $(15W_{i1} - (W_{i2})^2 + \sin(W_{i3}))$. Three functional forms of $f(\mu_i, \tau_t)$: additive $\mu_i + \tau_t$, multiplicative $\mu_i * \tau_t$, and cyclical $sin(\mu_i) * cos(\tau_t)$. In each combination, we set $N = 3{,}000$, $T = 90$, $J = 100$.

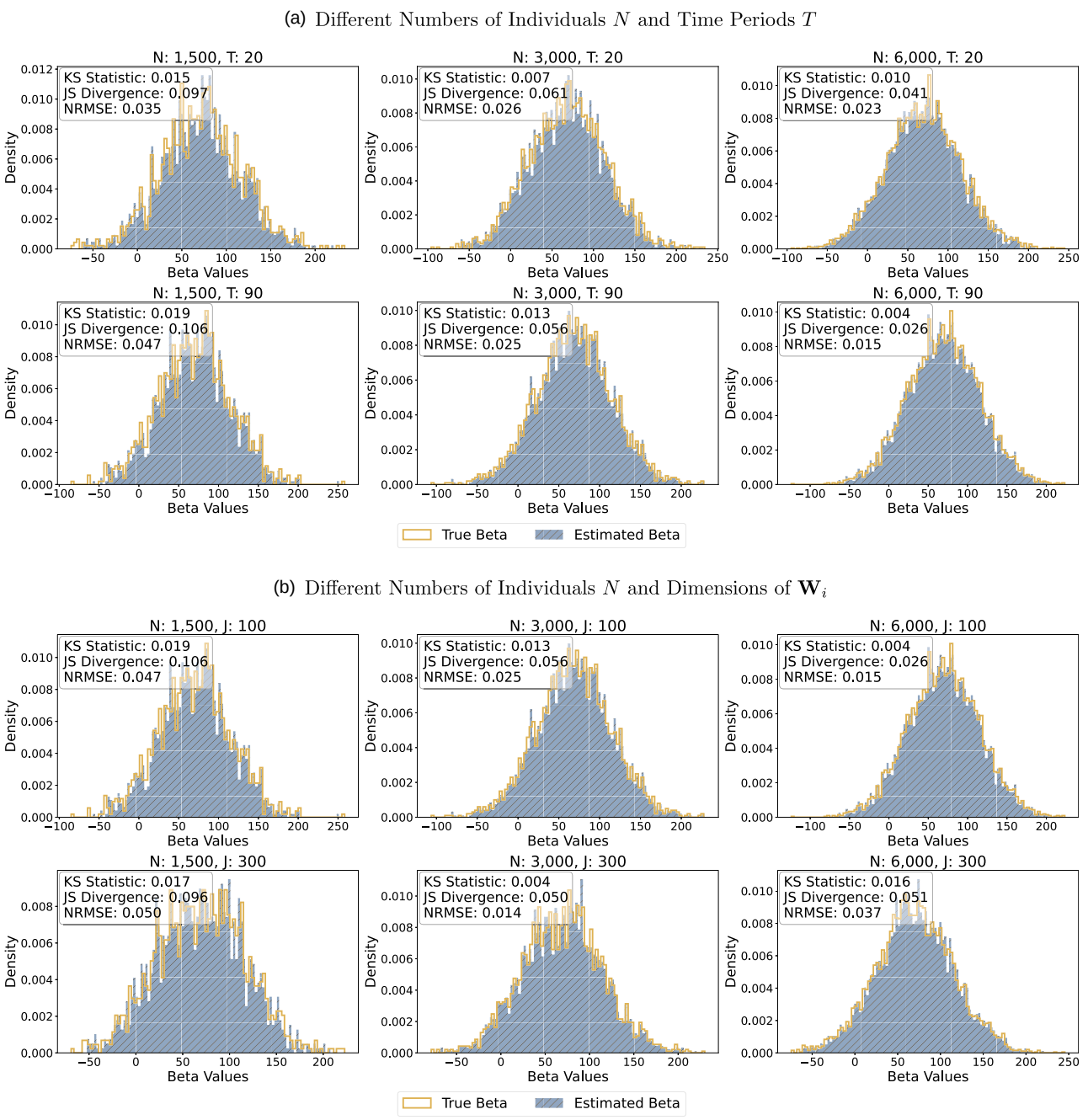
There are $2 \times 3 = 6$ combinations in total. The detailed specifications are listed in the note of Figure 2. The results demonstrate that the true β_is can be estimated accurately across different functional forms of the heterogeneous treatment effect $\beta(\cdot)$ and fixed effects $f(\cdot,\cdot)$.

Second, we test the performance of our approach under different sample sizes (N, T) and feature dimensions (J). In Figure 3(a), we vary the number of individuals $N \in \{1{,}500, 3{,}000, 6{,}000\}$ and the number of time periods $T \in \{20, 90\}$, with J fixed at 100. The estimation is fairly accurate for all the combinations. A general trend is that when the number of individuals increases (from left to right) or when the number of time periods increases (from top to bottom), the estimation becomes more accurate. Note that for the case of $(N = 1{,}500, T = 20)$, we increase the number of epochs to 20 to achieve satisfactory performance, whereas the number of epochs is set to 10 in other cases. This suggests that a higher number of epochs is required when working with fewer observations. In Figure 3(b), we vary the number of individuals $N \in \{1{,}500, 3{,}000, 6{,}000\}$, and the dimension of pretreatment features $J \in \{100, 300\}$, with T fixed at 90. The estimation performance improves when N becomes larger but is not sensitive to the change in J.

Third, we examine what happens if there are two different groups of individuals drawn from different distributions. In particular, we assume half of the population is a high-performing population, with each element in $\mathbf{W}_i$ drawn from $N(8,3)$ and individual fixed effects μ_i drawn from $N(2,5)$; the other half of the population is low-performing, with each element in $\mathbf{W}_i$ drawn from $N(5,3)$ and individual fixed effects μ_i drawn from $N(0,5)$. We assume that each individual i is treated with probability $\Pr(D_i = 1) = 1/[1 + \exp(-MinMax(W_{i1} + W_{i2} + W_{i3} + \mu_i))]$, where $MinMax(\cdot)$ means a min-max normalization function. Note that the assumed $\Pr(D_i = 1)$ increases with $W_{i1} + W_{i2} + W_{i3} + \mu_i$, so the treated group will consist of more individuals from the high-performing population, and the control group will consist of more from the low-performing population. This reflects the case when there is systematic difference between the treated and control groups, and the difference is contributed by both observable and unobservable features.[9] As shown in Figure 4(a), our model estimation can still uncover the true distribution of β_is in this case. In addition, we consider the case when there is also a shift in time fixed effects τ_t, which reflects time-variant confounders that affect all individuals equally or randomly. Specifically, we assume that τ_t follows $N(0,1)$ before treatment and $N(2,1)$ after treatment. The model estimation still performs well, as shown in Figure 4(b).

Fourth, we examine the performance of our approach across different implementation choices in the estimation procedure: the optimizer {Adam, SGD} (SGD stands for stochastic gradient descent), the number of notes in each layer of the DNN {32, 64}, the batch size {512, 1024}, the learning rate {0.001, 0.005}, and the number of epochs {5, 10}.[10] Table 2 shows the estimation performance (i.e., KS statistic, JS divergence, and NRMSE, all calculated on the test set) under different parameter values. In each row, the reported values are averaged across 10 simulations.

Figure 3. (Color online) Estimation Performance of Deep-DiD Under Different Sample Sizes and Feature Dimensions



Notes. In both subfigures, the true functional form of $\beta(\mathbf{W}_i)$ is $(15W_{i1} - 2W_{i2} + W_{i3})$. The true functional form of $f(\mu_i, \tau_t)$ is $\mu_i * \tau_t$. In (a), we set $J = 100$. In (b), we set $T = 90$. The number of epochs is set to 20 when $N = 1{,}500$, $T = 20$ in (a) and 10 in all the other cases.

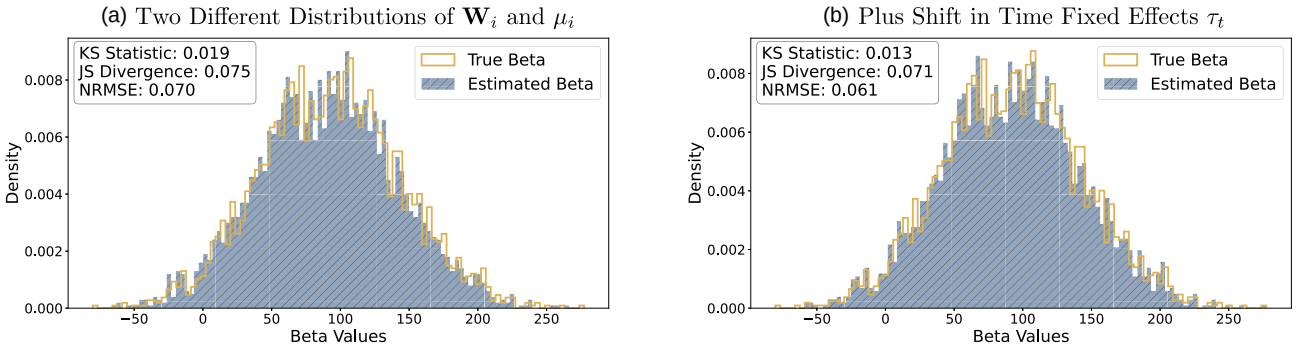
We find that the SGD optimizer does not work well in our simulation settings. When the optimizer is Adam, the model estimation performs well, and the estimation performance is not very sensitive to other implementation choices.

Deep-DiD vs. Linear Model and Random Forests. Next, we compare the performance of Deep-DiD method versus representative existing approaches.

We first show that our method has more robust performance than a conventional parametric approach; that is, if we do not use DNN but rather assume a specific functional form (e.g., a linear model) for $\beta(\mathbf{W}_i)$ or $f(\mu_i, \tau_t)$ in the estimation, the estimated distribution of β_is can be largely biased.

We first test what happens if we assume that $\beta(\cdot)$ is a linear function (see the Figure 5(a), top row).[11] When the true $\beta(\mathbf{W}_i)$ is a nonlinear function of $\mathbf{W}_i$, the

Figure 4. (Color online) Estimation Performance of Deep-DiD with Observable Differences and Unobservable Confounders



Notes. In both subfigures, the true functional form of $\beta(\mathbf{W}_i)$ is $(15W_{i1} - 2W_{i2} + W_{i3})$. The true functional form of $f(\mu_i, \tau_t)$ is $\mu_i + \tau_t$. $N = 3{,}000$, $J = 100$, $T = 90$.

estimated distribution of β_is is largely different from the true distribution (Figure 5(a), top right). Surprisingly, even when the true $\beta(\mathbf{W}_i)$ is a linear function of $\mathbf{W}_i$ (but includes only some elements of $\mathbf{W}_i$ in the function), the linear model estimation performs poorly (Figure 5(a), top left). We find that only when the true $\beta(\mathbf{W}_i)$ is linear and includes all elements of $\mathbf{W}_i$ (called the "linear-all" case) does the linear model accurately capture the true value of β_is (Figure 5(a), top middle). The contrast between the linear and linear-all cases indicates that linear models are poor at variable selection; that is, they cannot accurately identify which pretreatment features' variations contribute to the variations in β_is. This explains why linear models may yield inaccurate estimates even when the true functional form is linear. In contrast, when we use DNNs to estimate $\beta(\cdot)$ (in Figure 5(a), bottom row), the estimated values of β_is closely approximate the true values, no matter whether the true functional relationship is linear (bottom left), linear-all (bottom middle), or nonlinear (bottom right).

In Figure 5(b), we test what happens if we assume that $f(\cdot,\cdot)$ is a linear function in estimation. When the true $f(\cdot,\cdot)$ is linear, which means that the true individual fixed effects and time fixed effects are just additive, the linear model works and performs as well as using DNN to estimate the nuisance function (Figure 5(b), left column). When the true $f(\cdot,\cdot)$ is nonlinear—specifically multiplicative—the linear model yields biased estimates of β_i's (Figure 5(b), top right), whereas our DNN approach still uncovers the true β_i's effectively (Figure 5(b), bottom right).

To summarize, our Deep-DiD method exhibits remarkable adaptability, successfully capturing both linear and nonlinear relationships without requiring a prior specification of the functional form.

Furthermore, we compare DNN to random forests. Note that combining random forests with a DiD model with fixed effects is challenging and has not been solved as far as we know, and it is beyond the scope of this paper to address this issue.[12] Thus, we set aside the DiD setting for now and focus solely on functional approximation. The comparison between the estimated and true values of β_is is displayed in Figure 6, with the implementation details provided in the figure note. This figure shows that DNN outperforms random forests in uncovering true β_is, demonstrating DNN's superior performance in functional approximation compared with random forests.

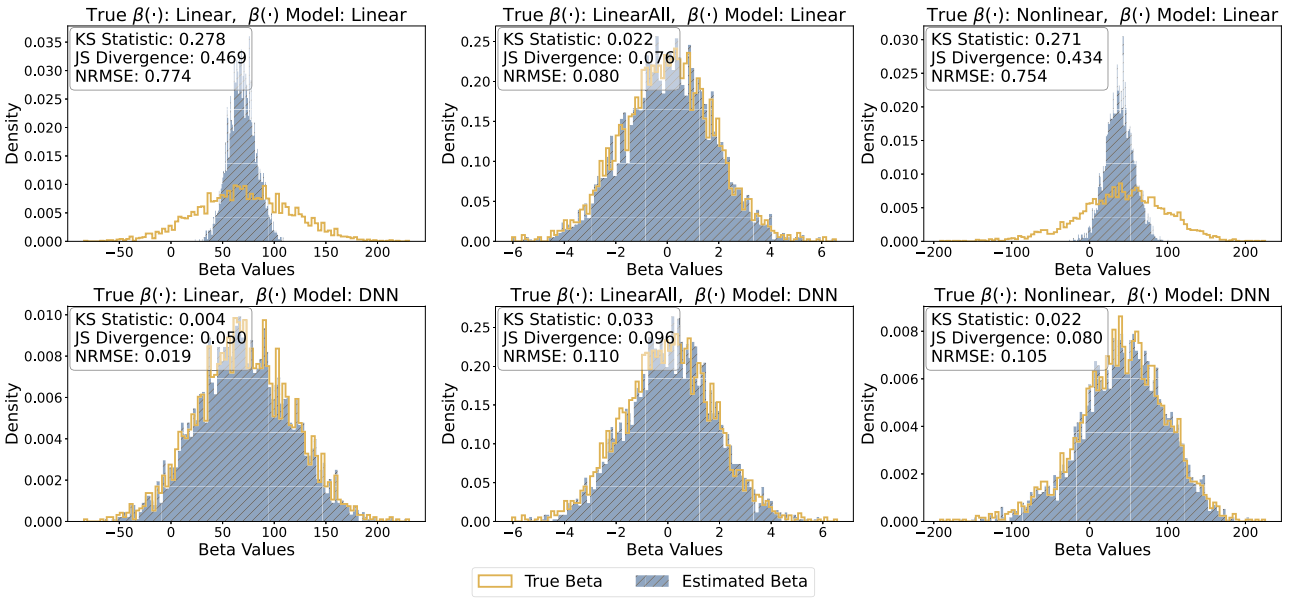
Table 2. Estimation Performance of Deep-DiD Across Different Implementation Choices

No.	Optimizer	No. of nodes in each hidden layer	Batch size	Learning rate	Epochs	KS statistic	JS divergence	NRMSE
1	Adam	32	512	0.001	10	0.015	0.071	0.061
2	Adam	64	512	0.001	10	0.015	0.077	0.085
3	Adam	32	1024	0.001	10	0.014	0.066	0.045
4	Adam	32	512	0.005	10	0.021	0.083	0.101
5	Adam	32	512	0.001	5	0.016	0.070	0.057
6	SGD	32	512	0.001	10	0.523	0.793	1.008
7	SGD	64	512	0.001	10	0.507	0.792	1.000
8	SGD	32	1024	0.001	10	0.529	0.792	1.021
9	SGD	32	512	0.001	5	0.535	0.793	1.023
10	SGD	32	512	0.005	10	0.903	0.958	1,263,449.794

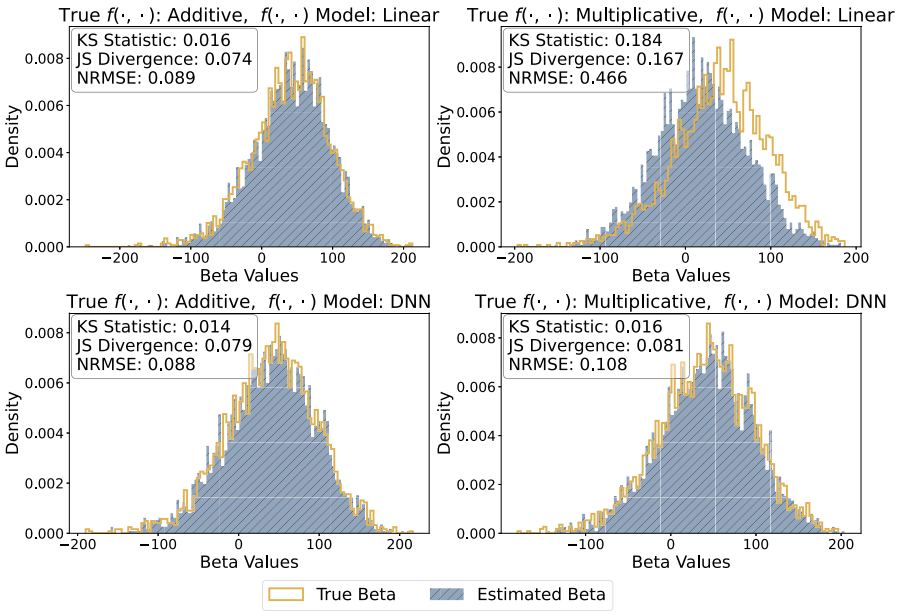
Notes. The true functional form of $\beta(\mathbf{W}_i)$ is $(15W_{i1} - 2W_{i2} + W_{i3})$. The true functional form of $f(\mu_i, \tau_t)$ is $\mu_i + \tau_t$. We set $N = 3{,}000$, $T = 90$, $J = 100$.

Figure 5. (Color online) Robustness of DNN: Compare DNN to Linear Model

(a) Compared DNN to Linear Estimator of $\beta(\cdot)$



(b) Compare DNN to Linear Estimator of $f(\cdot,\cdot)$

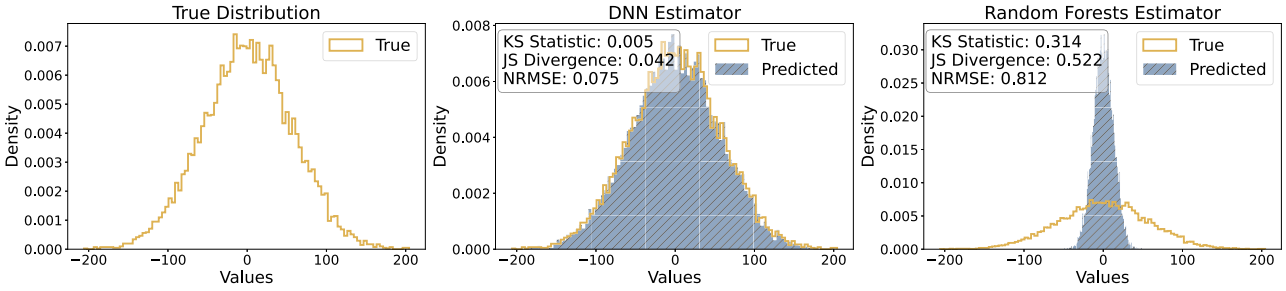


Notes. In all simulations, we set $N = 3{,}000$, $T = 90$, and $J = 100$. In (a), the true functional forms of $\beta(\mathbf{W}_i)$ are linear ($15W_{i1} - 2W_{i2} + W_{i3}$), linear-all ($\sum_{j=1}^{J} \frac{\alpha_j}{J} w_j$, where $\{\alpha_1, \ldots, \alpha_J\}$ is an arithmetic sequence decreasing uniformly from 10 to −10), and nonlinear ($15W_{i1} - (W_{i2})^2 + \sin(W_{i3})$). The true functional form of $f(\mu_i, \tau_t)$ is multiplicative ($\mu_i \tau_t$). In (b), we generate each μ_i following $N(5,1)$ and each τ_t following $N(-5,1)$. The true functional forms for $f(\mu_i, \tau_t)$ are additive ($\mu_i + \tau_t$) and multiplicative ($\mu_i \tau_t$). The true functional form of $\beta(\mathbf{W}_i)$ is $15W_{i1} - (W_{i2})^2 + \sin(W_{i3})$.

Compared with random forests, DNN not only has superior performance in functional approximation but also has the following advantages in estimating HTE. First, DNN can be implemented on production-level deep learning platforms like PyTorch. This highly mature platform guarantees stability and scalability, making the processing of extensive data and large number of covariates notably seamless. Second, as mentioned above, it is still yet unknown to us how the generalized random forest approach can deal with large-dimensional fixed effects.

2.1.3. Discussion. Note that the findings above are based on a well-defined simulation setting where

Figure 6. (Color online) Compare DNN to Random Forests



Notes. We simulate a data set with $N = 50{,}000$ and $J = 100$. W_{ij}'s $(i = 1,\ldots,N, j = 1,\ldots,J)$ are i.i.d., following a uniform distribution $U(-10,10)$. The true functional form of $\beta(\mathbf{W}_i)$ is $\beta(\mathbf{W}_i) = 5\sin(W_{i1}) + 2\cos(W_{i2}W_{i3}) + 3\exp(-((W_{i4}-2)^2 + (W_{i5}+1)^2)/10) + \sum_{j=6}^{100} W_{ij}$. Other specifications are the same as that described in Section 2.1.1. We train the DNN and the random forest models on the training set to get an estimated relationship $\hat{\beta}(\mathbf{W}_i)$ and then evaluate the model performance by comparing the estimated $\hat{\beta}_i$ with the true β_i in the test set. For the random forest model, we use the RandomForestRegressor from scikit-learn with 100 trees. For the DNN model, we implement a Multi-Layer Perceptron (MLP) in PyTorch with two hidden layers of 128 and 64 neurons, with both using ReLU activation functions. The model is trained for 20 epochs.

theoretical expectations are closely met. Real-world scenarios often introduce more complexities and nuances, which may require adjustments of the architecture and hyperparameters of DNNs, including those that showed less sensitivity in our simulations.

One may wonder, in a real-world application (rather than a simulation task) where the true treatment effects are unobservable, how we can assess the model's performance and, furthermore, how to select the architecture and hyperparameters for the DNNs in the model. It is important to note that we are not training the model using observed treatment effects but rather using observed Y_i's. Essentially, our method estimates the individual-level treatment effect β_i as a function of individual pretreatment features $\mathbf{W}_i$ based on the equation $\beta_i = \mathrm{E}[Y(1)|\mathbf{W}_i] - \mathrm{E}[Y(0)|\mathbf{W}_i]$, where $Y(d) \equiv Y(D_i = d), d = 0, 1$. For each individual i, only one of the $\{Y(1), Y(0)\}$ is observed because he or she was either treated or untreated, and the other's expected value (as a function of $\mathbf{W}_i$) is predicted by the model. More specifically, we learn the relationship between $Y(d)$ and $\mathbf{W}_i$ based on the individuals whose $Y(d)$ are observable and then predict $\mathrm{E}[Y(d)|\mathbf{W}_i]$ for individuals whose $Y(d)$ are unobservable $(d = 0, 1)$. Therefore, although the final output is an estimated causal effect, our method fundamentally remains a prediction task. As with other prediction problems, performance can be evaluated using measures such as MSE, which quantify the discrepancy between predicted $\hat{Y}_i$s and actual Y_is observed. The way of choosing the DNN architecture and hyperparameters is similar to other prediction tasks. Existing literature offers practical guidelines for these choices (see, e.g. Bengio 2012, Goodfellow et al. 2016, and Aggarwal 2018). Architecture selection typically starts from matching the architecture to the data modality: Multi-Layer Perceptrons (MLPs) for tabular data, Convolutional Neural Networks (CNNs) for images, or Recurrent Neural Networks (RNNs) for sequential data. The selection of network depth, width, activation functions, and training configurations such as the optimizer (e.g., Adam, SGD) and learning rate usually requires systematic experimentation. In this process, the training and validation losses are closely monitored to diagnose issues like overfitting or underfitting. A simple baseline often serves as a good starting point. In the context of this paper, a baseline MLP with two to three hidden layers has proven very effective, as shown in our simulations and empirical application (Section 4).

3. Empirical Background

We apply our Deep-DiD method to a real-world empirical setting to estimate the HTEs of a specific treatment. We collaborate with a large short-video sharing platform that was expanding their global user base. As part of its global growth strategy, the platform launched a "Creator Signing Program" in three countries located in South America, Africa, and Europe. The program aimed to incentivize content creators to upload more high-quality original content to the platform. Selected creators were able to sign a contract with the platform to join the program.[13] After joining the program, they received monthly payments by the platform based on their performance during the month. We apply our approach to estimate the HTEs of this signing program and show how our estimation results can help the platform optimize its creator selection.

We focus on creators from the three countries who were signed from May 1, 2022, to June 30, 2022.[14] Because there is a very large number of unsigned creators—far too many to include all in our analyses—we construct a control group from the pool of unsigned creators (as of data collection) by matching them with signed creators. Our matching procedure is unconventional in the sense that we match on creators' trends of outcome measures rather than their

mean levels. A conventional matching procedure selects untreated individuals who are similar to treated individuals based on observable characteristics, aiming to construct a comparable control group such that the treatment assignment is as good as random. After a conventional matching procedure, treatment effects can be estimated by directly comparing posttreatment means between treated and control groups. In our setting, however, signed creators are systematically different from unsigned creators; they generally demonstrate higher performance measures even before the treatment. Thus, constructing a control group that is statistically indifferent from the treated group through conventional matching is challenging.[15] Nevertheless, DiD requires only that the treated and control groups exhibit parallel trends in outcome measures before treatment, not that they be statistically indifferent before treatment.[16] Therefore, we match on the trends of outcome measures, aiming to construct a control group that exhibits parallel pretreatment trends with the treated group. In particular, for each creator who uploaded at least five videos[17] from February 1 to April 30, 2022, we estimate the trend of each performance measure using a linear regression. Because signed creators have different signing dates (after May 1) and unsigned creators do not have signing dates, we use data in the common period, February 1 to April 30, to estimate the trends. Then, we conduct propensity score matching (PSM), with the estimated trends as matching variables. More details of the matching procedure are available in Online Appendix A.1.[18]

The matched data set comprises 2,343 creators in the signed group and 2,343 creators in the control group. In Online Appendix Table A.2, we test that the signed group and the control group have parallel pretreatment trends for all outcome measures. In particular, we regress the creator-day level outcome measures on *Treated* × # *Days to Signing Date*, with creator fixed effects and day fixed effects controlled for. Part A of the table uses observations from May 1, 2022, to each creator's signing date. Because the matching procedure uses the data of February 1–April 30, 2022, using the data from May 1 to the signing date to test the parallel trend is an "extrapolation" test," which can be considered as a test of the assumption that the treatment and control groups would have followed the parallel trajectories over time in the absence of treatment. Part B of the table uses an extended period from February 1, 2022, to each creator's signing date. Both Part A and Part B show statistically insignificant coefficients for the interaction term *Treated* × # *Days to Signing Date* across all outcome measures, supporting the parallel-trend assumption.

For each creator, we designated their signing date as Day 0 and collected data of the 89 days before[19] and 90 days after the signing date (including the signing date). For control group creators, we defined their Day 0 as the signing date of their matched treated creator. The data we obtained included (1) creator features, like the number of followers a creator has and how many users a creator is following, the creator's gender, country, etc.; (2) creators' production, like the number of videos a creator uploads on a day; (3) video viewing, that is, the number of views, the number of completed views, and the total play duration each video has; (4) users' interactions (engagement behaviors), that is, the number of likes, comments, shares, and follows[20] each video has, and (5) visual and audio features of each video. We will elaborate more on these variables in Sections 3.1 and 4.1, when we define and construct the outcome variables and input variables.

3.1. Outcome Variable: Measurement of Creator Performance

To study the effect of the signing program on creators' performance and how the platform should select creators to sign, we first need to determine how to measure a creator's performance.[21] The goal of the signing program is to incentivize creators to contribute more videos with original content and higher quality, and these videos in turn can attract users to spend more time and have more interactions on the platform. These make the platform a prosperous content platform and attract more users in the long run. Therefore, when choosing the key performance measures, we focus on how to measure a creator's contribution to the platform in these aspects. Based on this principle and communication with managers in the company, we choose two key performance measures, as illustrated below.

The first performance measure is *User Time Contributed*, meaning the cumulative time length that the creator's videos have been watched by all viewers. This measure is important to the platform because it captures how much user screen time the video contributes to the platform, which is directly related to the platform's profitability. The platform cares not only about how much time users spend on the platform but also about whether users get engaged and interact on the platform, which can be measured by how many likes, comments, shares, and follows a video gets. These responses reflect different degrees of user engagement (Brodie et al. 2011, Oestreicher-Singer and Zalmanson 2013, Yang et al. 2019). To offer a holistic measure of engagement, we integrate these four engagement measures by performing Principal Component Analysis (PCA) and take the first component as the *User Engagement* measure. Therefore, we determine two key performance measures of creators, *User Time Contributed* and *User Engagement*, both of which are defined at the creator-day level.[22] Table 3 presents the summary statistics of the two performance measures.

Table 3. Summary Statistics of Outcome Variables

	Mean	S.D.	Median	Min	Max	*N*
User time contributed	3,342.299	60,047.410	0	0	1.315e+07	838,794
User engagement	0.010	1.816	−0.112	−0.112	469.509	838,794

Notes. This table is based on the data of all 2,343*2 creators in the matched data set. Observations are at the creator-day level. Each creator has 179 observations: 89 days before the signing date and 90 days after the signing date (including the signing date). Notice that *User Engagement* has negative values because it is the first component of PCA, which has been normalized following the standard normal distribution.

Note that our approach does not hinge upon our choice of performance measures, and it can be applied easily when other measures are chosen as the outcome variable. For example, a platform may define a weighted sum of these two performance measures as the single key performance measure. Then they can easily calculate that measure and use it as the outcome variable when applying our method.

3.2. Impact of the Signing Program on Creator Performance

We first graphically show the effect of the signing program on creator performance. For each performance measure, Figure 7 shows its daily average value of creators in the signed group versus in the control group in the 89 days before and 90 days after Day 0.

We can see that the signed group has a significant jump in both performance measures after Day 0, and both performance measures are relatively stable in the 90 days after Day 0, implying that the positive impacts on *User Time Contributed* and *User Engagement* of signed creators tend to be long-lasting.

To quantify the impact of the signing program on creator performance, we conduct DiD analyses as suggested by model (1), as shown in Table 4. Columns (1) and (3) report the results when the Treated dummy (D_i) and After dummy (A_t) are included in the regression, and Columns (2) and (4) show the results when creator and day fixed effects are controlled for. For both performance measures, the coefficients of the interaction term are positive and statistically significant, and their magnitudes are relatively stable after we control for fixed effects.[23] The relative effect size of the treatment is larger than 100% for both performance measures. To summarize, the DiD regression results show that the signing program has a significant and substantial positive impact on signed creators' performance, which is measured by the amount of user watching time and engagement contributed by the videos.

4. Using Deep-DiD to Estimate the Heterogeneous Treatment Effects of the Signing Program

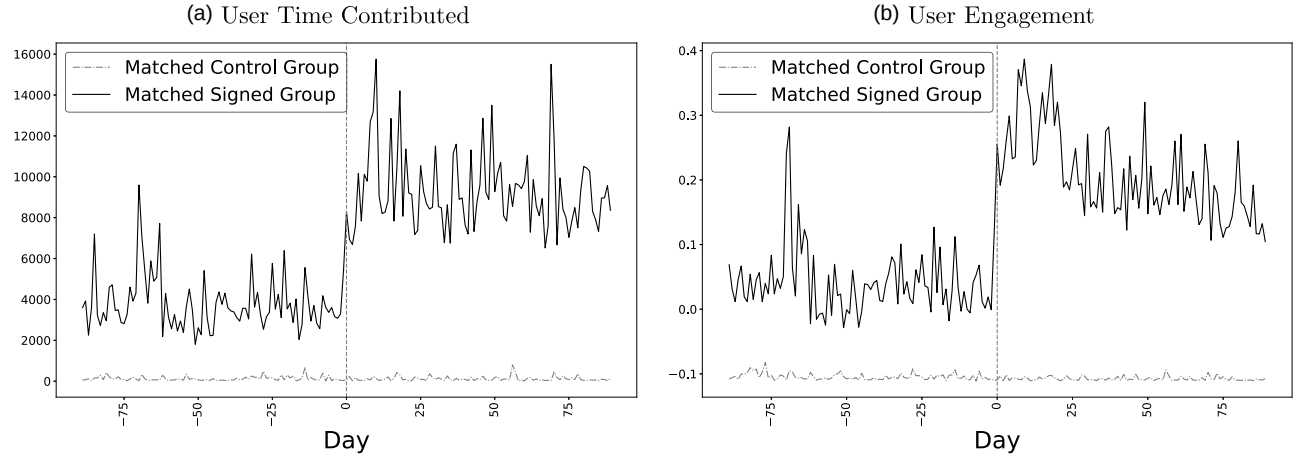
Now we apply the Deep-DiD method to the empirical setting to estimate the heterogeneous treatment effects of the signing program.

In Section 4.1, we describe how we construct creators' pretreatment characteristics $\mathbf{W}_i$. In Section 4.2, we provide the details of the model estimation techniques and estimation results.

4.1. Creators' Pretreatment Features

We construct the pretreatment features of creators $\mathbf{W}_i$, utilizing the rich data we have as much as possible. We not only make use of the structured data describing creators' production behaviors and viewers' responses but also utilize the video and audio features, which are derived from unstructured video data using

Figure 7. The Impact of the Signing Program on Creator Performance



deep-learning-based methods. We encountered some data loss in this process because some videos had been deleted by creators and become unavailable for downloading. We remove the creators who have more than 10% of videos missing. After this step, we are left with 2,204 creators, among which 1,126 are from the treated group and 1,078 are from the control group. The results of conventional DiD analyses based on this reduced data set (see Online Appendix A.3) are very close to the full sample results as shown in Figure 7 and Table 4.

In the following, we describe how we construct variables in $\mathbf{W}_i$. We classify the creator features into three categories: (1) creators' production behavior and viewers' responses, (2) visual features, and (3) audio features.

For the first category, *creators' production behavior and viewers' responses*, we include variables that describe a creator's production quantity (*Production*, meaning the number of videos uploaded on the day), viewers' viewing behavior (*Num of Plays*, meaning how many times the creator's videos were played by viewers on the day; *Num of Complete Plays*, meaning how many times the creator's videos were watched by viewers from the beginning to the end on the day; and *Play Duration*, meaning the total time length that the creator's videos were watched by viewers on the day), and viewers' interactions with the creator (*Num of Likes*, *Num of Comments*, *Num of Shares*, *Num of Follows*, meaning the total number of likes, comments, shares, and follows the creator's videos get on the day). We also include variables that describe the creator's account status on the platform: how many followers the creator has on the day (*Num of Followers*) and how many users the creator is following on the day (*Num of Following Users*). To reduce the dimension of $\mathbf{W}_i$, we aggregate these variables from the creator-day level to the creator level by taking the mean, median, minimum (min), maximum (max), and standard deviation (SD) of each variable for each creator, using observations in the 89 days before the signing date. Furthermore, to capture the trend of changes, we also calculate the slope of each variable for each creator to include in $\mathbf{W}_i$. The slope of a variable captures whether and by how much this variable is increasing, decreasing, or remaining relatively steady for a creator.

A video can be considered as a combination of a large number of consecutive images plus audio. The manner in which the content is visually and aurally presented to the audience can directly impact users' watching experience and thus can influence their willingness to continue watching and get engaged (see, e.g., Li et al. 2019, Hwang et al. 2021, Wang et al. 2021, Zhou et al. 2021, Cheng and Zhang 2024, and Yang et al. 2025). Having access to the videos uploaded by creators, we are able to extract visual and audio features from them to be included in $\mathbf{W}_i$. For *visual features*, we use eight visual quality measures, including *Blur Probability*, *Blocky Probability*, *Dirty Lens Probability*, *Noise Probability*, *Colorfulness*, *Defocus Probability*, *Aesthetic Score*, and *Quality Score*, as well as one similarity measure for each video. For *audio features*, we include five measures, which are *Audio Quality*, *Stereophonic Richness*, *Background Noise Probability*, *Dialog Probability*, and *Music probability*. For each visual feature and audio feature, we aggregate these video level measures to the creator level as the model input; for each creator and each measure, we take the mean, median, min, max, and SD of the measure on all the videos uploaded by the creator in the 89 days before the signing date and include these five summary statistics in $\mathbf{W}_i$.

In Online Appendix A.4, we provide more detailed explanations of the variables mentioned above, why we choose them, and how we calculate them as well as their summary statistics.

Table 4. The Impact of the Signing Program on Creator Performance: DiD Analyses

	Y: User time contributed		Y: User engagement	
X	(1)	(2)	(3)	(4)
Treated × *After*	5,458.946***	5,465.047***	0.1659***	0.1661***
	(719.306)	(727.049)	(0.0195)	(0.0198)
Treated	3,716.215***		0.1481***	
	(556.457)		(0.0182)	
After	−5.5291		−0.0030	
	(23.9079)		(0.0021)	
Creator FE		Yes		Yes
Natural day FE		Yes		Yes
Days to signing date FE		Yes		Yes
Relative effect size	146.90%		112.02%	
N	838,794	838,794	838,794	838,794
Adj. R^2	0.0039	0.1823	0.0050	0.1615

Notes. User Time Contributed is in minutes. "FE" stands for fixed effects. "Natural Day FE" means dummies represent the natural day (e.g., May 1, May 2…), whereas "Days to Signing Date FE" are dummies representing the number of days that the current day is before (negative value) or after (positive value) the signing date. The relative effect size is calculated using the coefficient of the interaction term divided by the coefficient of *Treated* dummy. Standard errors are clustered at the creator level and are shown in the parentheses.

***$p < 0.01$; **$p < 0.05$; *$p < 0.1$.

4.2. Model Estimation and Results

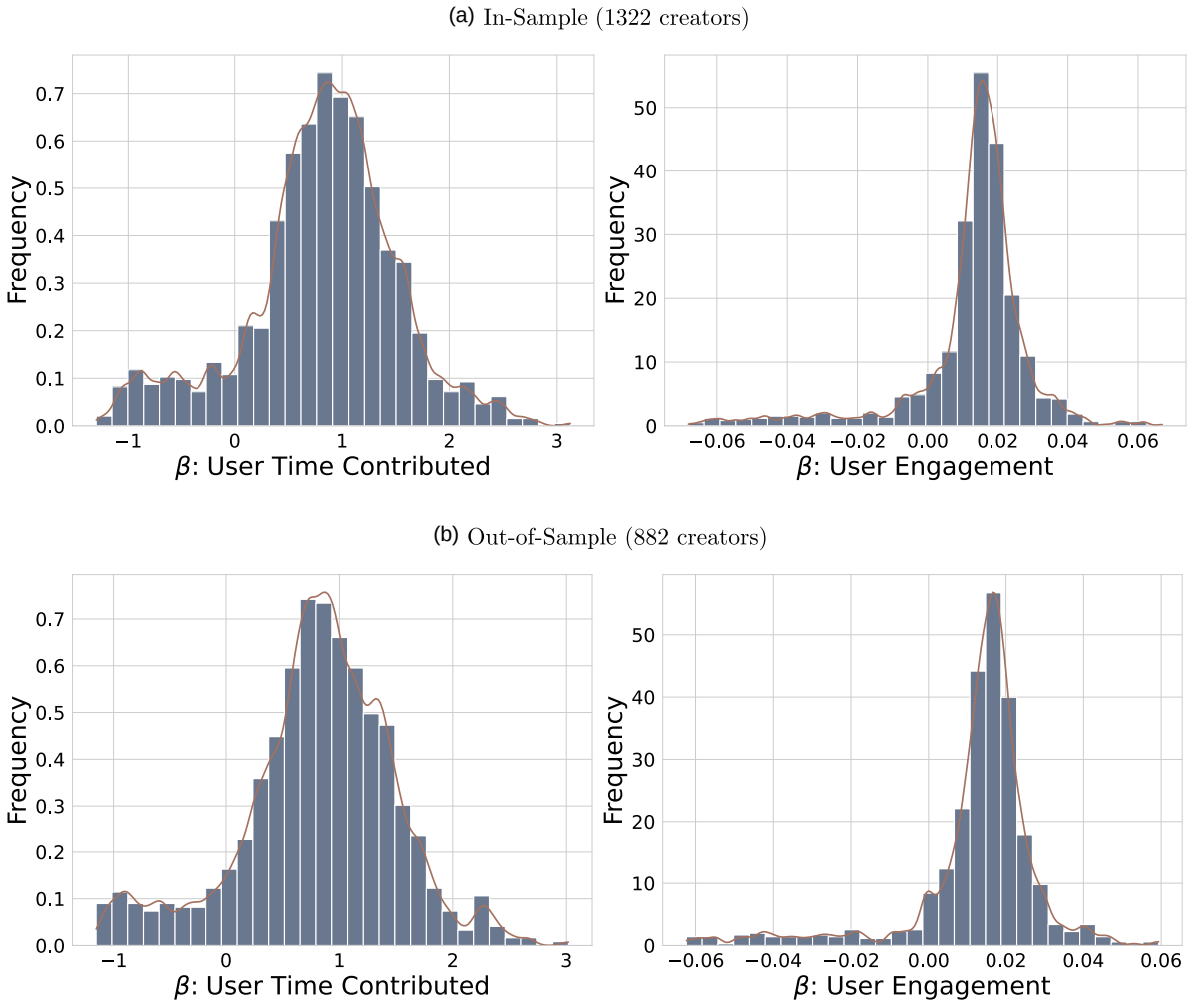
We apply the Deep-DiD method to estimate the HTE of the signing program on creators' performance. The implementation is similar to what we describe in Section 2.1.1, with some differences that are due to the complexities in data from a real-world setting. We describe the key techniques below.

Because many of the variables in pretreatment features $\mathbf{W}_i$ can be largely skewed or have outliers, feeding them to the input layer of the DNN may result in gradient explosion. Therefore, before feeding $\mathbf{W}_i$ into the model, we preprocess the data using winsorization,[24] log-transformation, and min-max normalization methods. These preprocessing steps are commonly used in deep learning, and they help to avoid gradient explosion and ensure a smooth convergence. More details of the data-preprocessing procedures are available in Online Appendix A.5.

We randomly split our data into two parts, 60% (1,322 creators) for model training and validation and 40% (882 creators) for out-of-sample testing. Within the first part, 80% of the observations are for model training, and 20% are for validation.[25] The loss function we use is the mean squared error (MSE). In the training process, we monitor the reported loss on the validation set to prevent overfitting. We use the Adam optimizer and adopt a learning rate of 0.001. We choose the batch size as 512 and the number of epochs as 20. We design three fully connected hidden layers of (128, 64, 32) neurons activated by the ReLU function. Note that in the model training process, the number of individuals $N = 1,322$, the number of pretreatment features $J = 130$, and the total time periods $T = 179$. According to our simulation results shown in Section 2.1.2, with this size of data, the Deep-DiD model is able to uncover the true distribution of β_is with fairly high accuracy under proper implementation choices.

As illustrated in Section 2.1.1, the trained model provides us with an estimated function $\hat{\beta}(\cdot)$. We can apply it to calculate $\hat{\beta}_i = \hat{\beta}(\mathbf{W}_i)$ for each i in the training-validation set and the test set. To mitigate the randomness in the model training process, we perform the model training 10 times, each time initialized with a different random seed. For each i, we take the average value of $\hat{\beta}_i$ across these 10 rounds as the final estimated value of $\hat{\beta}_i$.[26] The convergence plot in one of the 10 rounds is presented in Online Appendix A.5. Figure 8 shows the in-sample and out-of-sample

Figure 8. (Color online) Distribution of Estimated Heterogeneous Treatment Effects $\hat{\beta}_i$s



distributions of the estimated heterogeneous treatment effects $\hat{\beta}_i$s, which are averaged over 10 rounds, for the two performance measures. We also present the distributions of $\hat{\beta}_i$s from one of the 10 rounds in Online Appendix A.5, Figure A.7. The similarity between the distributions shown in Figure 8 and those in Figure A.7 demonstrates the robustness of our estimation. The distributions of $\hat{\beta}_i$s for both performance measures exhibit substantial heterogeneity across creators. The two distributions also exhibit substantial dissimilarities, indicating that they capture distinct aspects of creators' performances.

5. Creator Selection

Now we show how our method can help the platform identify the right creators to sign and whether it outperforms platform selection. The idea is straightforward. The goal of signing a creator is to incentivize the creator to improve his or her performance. Our model estimation gives $\hat{\beta}(\mathbf{W}_i)$, that is, the estimated treatment effect of the signing program on each individual creator's performance as a function of his or her pretreatment features $\mathbf{W}_i$. Given a new set of creators, the platform can calculate their estimated treatment effects $\hat{\beta}_i$s based on our estimated function $\hat{\beta}(\cdot)$ and their pretreatment features $\mathbf{W}_i$s. Suppose the platform plans to select K creators out of this set of creators; it would then simply select the top K creators with the highest $\hat{\beta}_i$s.

5.1. Creator Selection by Deep-DiD Model vs. Platform Selection

We test the performance of our estimated model on creator selection using the test set (i.e., out-of-sample). Among the 882 creators in the test set, 441 of them are from the signed group. We label them as *Platform Selected*. To compare the performance of our method to that of platform selection, we identify the same number (441) of creators with the highest β_is from the 882 creators in the test sample and label them as *Model Selected*. Here, we set the number of Model Selected creators the same as that of the Platform Selected ones so that the comparison between the two groups is fair. The fewer creators we select, the higher the selection bar (in terms of the estimated treatment effects) can be. Because we have no access to information regarding the exact cost and monetary benefit of signing a creator, we cannot determine the optimal number of creators to sign.

Among the 441 *Model Selected* creators based on $\hat{\beta}_i$s when the performance measure is *User Time Contributed*, 248 are from the signed group. When the performance measure is *User Engagement*, among the *Model Selected* creators, 225 are from the signed group. For both performance measures, nearly 50% of the *Model Selected* creators were not selected by the platform.

As a first evaluation, we compare the average estimated treatment effects (the mean of $\hat{\beta}_i$s) of *Model Selected* versus *Platform Selected* creators. As shown in Figure 9(a), for both performance measures, the *Model Selected* creators have substantially higher average estimated treatment effects than the *Platform Selected* creators.

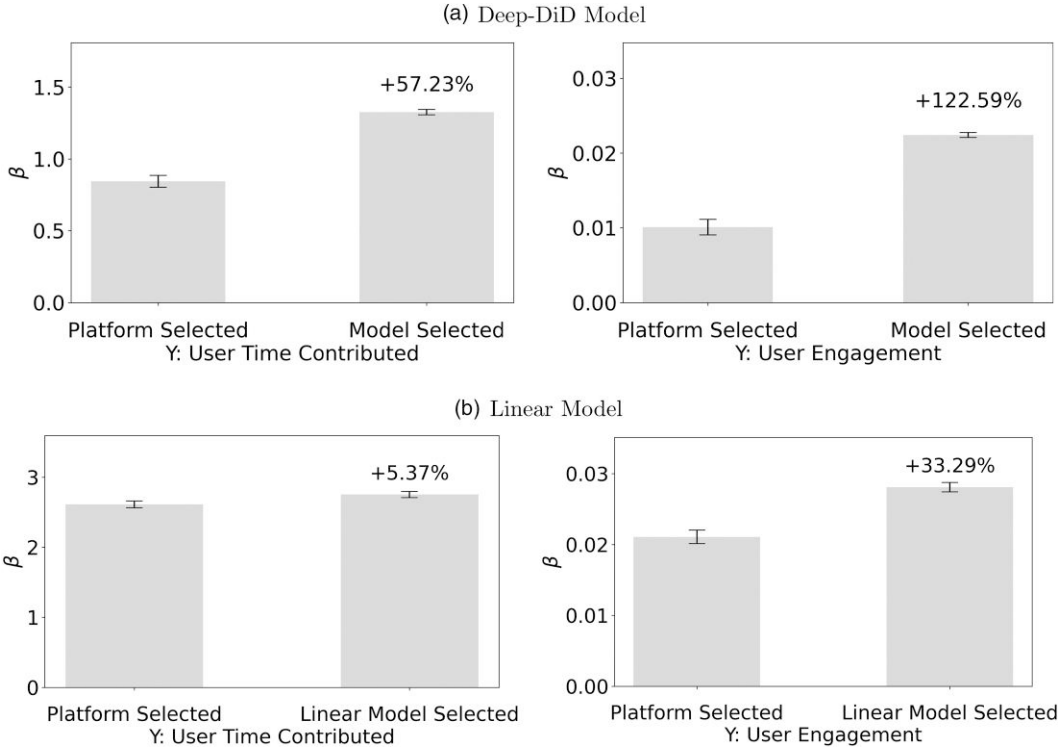
One may argue that the comparison above is not a fair comparison because *Model Selected* creators are selected based on $\hat{\beta}_i$s, and thus, it is not surprising that they have higher average $\hat{\beta}_i$s than *Platform Selected* creators, although the magnitude of the difference cannot be predicted ex ante. To resolve this concern, we come up with a second way to evaluate the performance of our method. For each creator who was actually signed, we can observe how much his or her performance changed after he or she got signed. We define the creator's average daily performance after the signing date minus that before the signing date as the creator's *Performance Jump*, which can be considered as the empirically observed "treatment effect" of the creator. The *Performance Jump* can also be negative if the creator's average daily performance decreased. We divide the 441 signed creators in the test set into two groups:[27] those who are also selected by the model and those who are not selected by the model. We calculate the average performance jump (APJ) for each of these two groups as well as for all 441 signed creators (see Table 5, Part A). We calculate the percentage difference between the APJ of creators selected by the model versus that of all signed creators. For both performance measures, the APJs of those selected by the model are substantially larger than those not selected by the model. The percentage difference in APJ between creators selected by the model and all signed creators is above 70% for both performance measures. This evaluation provides further empirical evidence for the superior performance of our method compared with platform selection. That is, the *Model Selected* creators not only have higher estimated treatment effects of the signing program, their actual increase in performance after being signed is also significantly larger than the platform selected ones.

In Online Appendix A.6, we conduct in-sample tests showing that among the 1,322 creators in the training-validation set, Model Selected creators also outperform Platform Selected creators.

5.2. Creator Selection by Deep-DiD Model vs. Linear Model

To provide another comparison benchmark for the creator selection based on our Deep-DiD model, we also conduct creator selection based on a linear model. That is, we assume that $\beta(\mathbf{W}_i) = \sum_{j=1,\ldots,J}\gamma_j W_{ij}$, and we estimate $\beta_i = \beta(\mathbf{W}_i)$ using the training-validation set.

Figure 9. Average Estimated Treatment Effects of Model Selected Versus Platform Selected: Out-Sample



Notes. In both (a) and (b), the Platform Selected group and the (Linear) Model Selected group consist of 441 creators from the test set. Each bar represents the mean value of $\hat{\beta}_i$s in the group, and the error bar represents the standard error of the mean.

Then, we apply the estimated linear model to the test set and identify 441 creators with the highest estimated treatment effects $\hat{\beta}_i$s, labeled as "Linear Model Selected." In Figure 9(b), we compare the average estimated treatment effects of the Linear Model Selected creators versus the Platform Selected creators. We can see that although the Linear Model Selected group has higher estimated treatment effects than the Platform Selected group by design, the percentage differences between the two groups are much smaller than those between the (Deep-DiD) Model Selected group and the Platform Selected group for both outcome measures (5.37% versus 57.23%, 33.29% versus 122.59%). This result indicates that, in expectation, the difference in treatment effects between the Model Selected creators and the Platform Selected creators is larger when the "model" is our Deep-DiD model rather than a simple linear model.

Table 5. Actual Performance Jump After Being Signed: Out-Sample

	User time contributed			User engagement		
Outcome Y	Selected by model (1)	Not selected by model (2)	All signed creators (3)	Selected by model (4)	Not selected by Model (5)	All signed creators (6)
Part A: Deep-DiD model						
Average performance jump	2.1379	0.0858	1.2259	0.2094	0.0284	0.1174
N	245	196	441	217	224	441
% Difference		74.394%			78.365%	
Part B: Linear model						
Average performance jump	1.1334	1.5966	1.2259	0.1605	0.0266	0.1174
N	353	88	441	299	142	441
% Difference		−7.545%			36.712%	

Notes. "All Signed Creators" means the 441 signed creators (i.e., Platform Selected) in the test set. We divide them into two groups: Selected by Model and Not Selected by Model. The model refers to the Deep-DiD model in part A and refers to the linear model in part B. The "% Difference" is the percentage difference in Average Performance Jump between the "Selected by Model" group and "All Signed Creators."

We also calculate the APJ for creators selected by the linear model among all signed creators in the test set and compare it to those not selected by the model as well as all signed creators. Results are provided in Table 5, part B. Comparing the percentage differences listed in part B with those in part A, we find that for both performance measures, our Deep-DiD model leads to a larger improvement in APJ than the linear model. When the outcome measure is *User Time Contributed*, the linear model even leads to lower APJ compared with all signed creators.

The comparisons above demonstrate the superiority of our Deep-DiD model versus the linear model. We can see that the Deep-DiD model performs consistently better than the linear model and demonstrates more robust performances across different outcome measures.

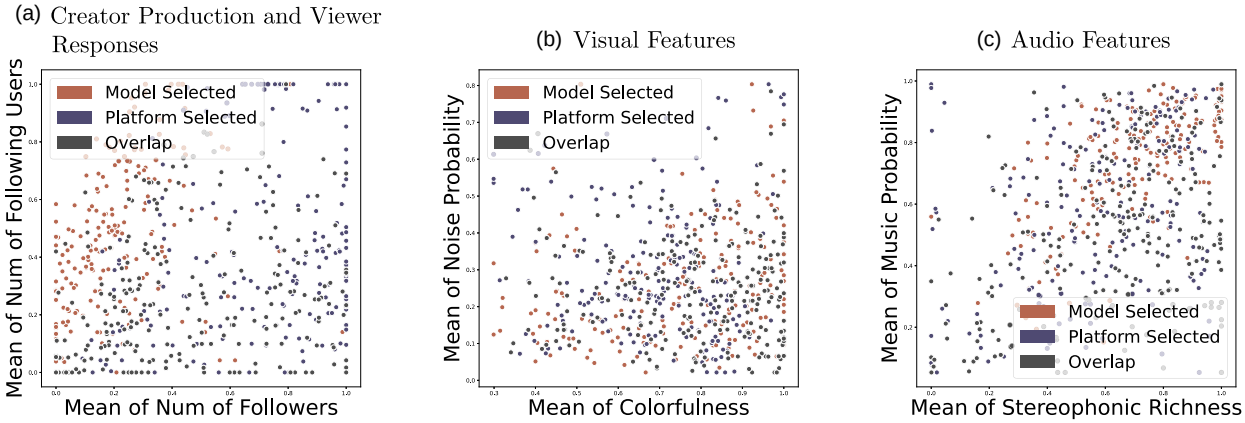
Note that the linear model here is estimated in a more modern way than the traditional ordinary least squares (OLS) method; we employ our Deep-DiD model framework but assume both the HTE estimation network and the fixed-effect network as single-layer (i.e., the input layer connects directly to the output layer without hidden layers). This is equivalent to assuming that $\beta(\cdot)$ is linear and the fixed effects are additive. It is mathematically equivalent to OLS because both methods minimize mean squared errors. However, compared with traditional OLS, estimating the linear model by training one-layer neural networks is much more computationally and memory efficient because the data are processed batch by batch and the model's weights are updated iteratively. This avoids the need to load the entire data set into memory and perform a single, massive matrix inversion, as what traditional OLS does. Therefore, our model framework can incorporate the linear model as a special case, and the estimation algorithm makes it possible to deal with large-scale, high-dimensional data. If estimating the linear model using traditional OLS, we are likely to encounter dimensional explosion because the model contains 130 three-way interaction terms $W_{ij} * D_i * A_t$ ($j = 1, \ldots, J$, where $J = 130$) as well as high-dimensional individual ($N = 1{,}322$) and time ($T = 179$) fixed effects. We may have to reduce the number of pretreatment features included in the model, which may further lower the performance of the linear model.

5.3. Visualization of Model Selected vs. Platform Selected Creators

Next we try to visualize how *Model Selected* creators may differ from those *Platform Selected* creators. As shown in Figure 10, we choose two representative dimensions in each feature category and visualize the two groups of creators on a two-dimensional plane. We use red dots to represent *Model Selected* creators, blue dots to represent *Platform Selected* creators, and gray dots to represent the overlapping creators (i.e., selected both by the model and by the platform). Here, we show only the comparison when the model selection is based on *User Time Contributed* as the outcome variable. The comparisons for the other outcome variable are delegated to Online Appendix A.7.

An interesting observation from Figure 10(a) is that human selection tends to select creators with a larger number of followers, whereas the model tends to select creators with a lower number of followers. However, in Figure 10, (b) and (c), we find it hard to observe a pattern in terms of whether *Model Selected* creators tend to be higher or lower in a certain feature. This highlights the advantages and importance of the DNN approach. The fact that it is hard to separate *Model Selected* creators and *Platform Selected* creators with a line on a two-dimensional plane implies that

Figure 10. Model Selected Versus Platform Selected Creators: *User Time Contributed* as the Outcome Variable



Note. When *User Time Contributed* is the outcome variable, among the 441 Model Selected creators in the test sample, 248 are from the signed (Platform Selected) group and 193 are from the control group.

there exist complicated interactions between creator features in determining the HTE. These complicated and ex ante unknown interactions are hard, if not impossible, to capture using parametric functions (not to say linear functions), whereas the DNN has superior performance in estimating such complicated nonparametric relationships.

6. Concluding Remarks

In this paper, we propose a Deep-DiD model that incorporates two DNN structures in a DiD framework to estimate HTEs. We conduct a series of simulations showing that our approach can uncover the true values of HTEs with high accuracy. Furthermore, we apply our method to an empirical setting to estimate the HTEs of a "Creator Signing Program" launched by a large video-sharing platform and use the estimated HTEs to optimize creator selection for the platform. We show that compared with creators selected by the platform, creators selected by our model have not only higher estimated treatment effects but also higher actual performance jumps after being signed. Creator selection based on the Deep-DiD model also demonstrates consistently better performance than that based on linear models.

To the best of our knowledge, our paper represents the first implementation of the DNN-based heterogeneity architecture proposed by Farrell et al. (2020) in a real empirical context. We innovatively integrate two DNN structures with the DiD framework, setting up a dual-network architecture to allow for flexible nonparametric relationships between the HTEs and pretreatment features, as well as possible interactions between fixed effects. We demonstrate the performance of this method through simulations.

Through the empirical application, we demonstrate the significant potential for managerial applications of our approach. Platforms or influencer agency companies can initiate a pilot signing program and utilize data from this program in conjunction with our Deep-DiD model to estimate the expected treatment effect of the signing program on individual creators' performance as a function of their pretreatment features. Then, they can automate the process of creator selection by targeting those with the highest estimated treatment effects.

The application of our method extends beyond this context. Our method can be easily tailored to other empirical settings, especially those with a rich data set, to estimate the HTEs of a certain treatment. Researchers can flexibly define the pretreatment features and the outcome variable. We have demonstrated that our method can uncover the true distribution of HTEs with high accuracy under various settings, and the estimated HTEs can help optimize the selection of individuals to be treated in the future.

A limitation of our method is that we cannot provide standard errors for the estimated treatment effects. It is still unknown whether the estimated individual-level heterogeneity has traditional consistency and asymptotic statistical properties (Farrell et al. 2020). Our focus is not on the statistical significance of treatment effects but on how they vary with individuals' pretreatment features, which enables targeting or selection based on pretreatment features, such as the creator selection application demonstrated in this paper or customer targeting for profit-maximizing (see, e.g., Simester et al. 2020 and Hitsch et al. 2024). Statistical inference is possible for functions of the estimated individual-level heterogeneity under certain conditions (Chernozhukov et al. 2018, Farrell et al. 2020), and thus, it is possible to derive standard errors for profits of targeting policies (Zhang 2024). This can be a future research direction.

Acknowledgments

The authors received helpful comments from conference attendees of the ISMS Marketing Science Conference, Hong Kong Quantitative Marketing Workshop, HKUST-USC Digital Transformation Conference, China India Insights Conference, Tsinghua SEM Alumni Forum, Marketing and the Creator Economy Conference, POMS-HK International Conference, INFORMS Annual Meeting, and seminar attendees at Tongji University, Shanghai University of Finance and Economics, and the Chinese University of Hong Kong. The authors thank the Senior Editor, the Associate Editor, and the anonymous reviewers for their constructive comments and helpful suggestions, which have greatly improved this paper. Yuhui Zhang was an employee of the company that provided the proprietary data set used in this paper. The first three authors contributed equally and are listed in reverse order of seniority.

Endnotes

[1] To protect the platform's identity, we do not reveal the exact countries. This platform was relatively new and did not have a large user base in these countries. Thus, instead of setting specific criteria and allowing eligible creators to apply, as YouTube and TikTok did, this platform handpicked a group of creators and offered them contracts.

[2] For creators in the control group, the signing date is defined as that of the creator in the treated group to whom they are matched.

[3] https://www.mckinsey.com/featured-insights/mckinsey-explainers/what-is-influencer-marketing.

[4] "Inference" here means to provide a point estimate as well as the standard errors of the estimate.

[5] In practice, we can first use the default embedding function in PyTorch to generate embeddings of the high-dimensional individual dummies and time-period dummies and use the embeddings as the input layer of this network. This method is equivalent to using the high-dimensional individual dummies and time-period dummies as the input layer directly, but it reduces the dimension of the

input layer of the DNN and thus can conserve RAM and decrease computation time.

[6] We make these distribution parameter assumptions so that there is enough variation in β_i and D_i, and the model performance under different settings can be demonstrated clearly. We have tested that our model also works under different distribution parameter assumptions.

[7] The learning rate is the step size by which parameters in the DNN are adjusted in the training process. The batch size is the number of data samples fed into the DNN each time when parameters in the DNN are updated. One epoch represents one complete forward and backward pass through all training samples. We observe the convergence plot and find that 10 epochs are enough for most of the simulation tasks in the sense that the validation loss curve becomes flat and starts to increase (Goodfellow et al. 2016). We increase the number of epochs for simulation tasks if we find that 10 epochs are enough. For example, in Figure 3(a), when $N = 1{,}500$, $T = 20$, we set the number of epochs to 20.

[8] We omit reporting p values for the Kolmogorov-Smirnov test for two reasons. First, there lacks a framework to quantify the theoretical variances or bounds of distributions estimated by neural networks with the current asymptotic theory, and therefore, the traditional statistical tests may not be applicable here. Second, our large sample sizes lead to consistent statistical significance, even for negligible differences. This is a well-known "large n problem" in hypothesis testing. Thus, we focus on the KS statistic and JS divergence as more informative measures of distributional differences rather than conducting hypothesis tests.

[9] Note that the unobservable time-invariant confounder μ_i is unobservable to researchers but can be observable to the platform when deciding the treatment assignment.

[10] We choose not to test different numbers of layers or activation functions because the structure we use in simulations (two hidden layers) already represents a minimal configuration in DNN, and ReLU is a common and effective choice for many neural network architectures.

[11] Note that when estimating $\beta(\cdot)$ using a linear model (in Figure 5(a)), we still use our Deep-DiD framework but assume the HTE estimation network as single-layer, which is equivalent to assuming $\beta(\cdot)$ as a linear function. To isolate the effect of the linear estimator of $\beta(\cdot)$, the fixed-effect network is still assumed to be a multi-layer DNN, consistent with other simulations. We discuss the advantage of this implementation over traditional ordinary least squares (OLS) in the final paragraph of Section 5.2. Similarly, when estimating the fixed effects $f(\cdot,\cdot)$ using a linear model (in Figure 5(b)), we use Deep-DiD framework with a single-layer, fixed-effect network, whereas the HTE estimation network remains a multilayer DNN as in other simulations.

[12] Random forests have been used to estimate heterogeneous treatment effects (see, e.g., Cong et al. 2021 and Guo et al. 2021), but we have not found the inclusion of fixed effects in existing works.

[13] The platform collaborates with several local agency companies to conduct the selection, and different local agency companies may adopt slightly different selection criteria. Therefore, unfortunately, we do not have access to their detailed selection criteria or the exact list of creators selected. We can only observe who joined the program. According to the platform manager, very few selected creators chose not to join the program because there was essentially no cost for them to do so and the contract was not exclusive. The monthly payment for a creator was determined based on a comprehensive evaluation of the creator's performance, including the number of videos created, the total user watching time and interactions contributed by the videos, etc. The exact calculation formula for the payment is confidential.

[14] Because of data confidentiality, we cannot reveal the total number of creators signed in this time period.

[15] It is possible to match a group of unsigned creators with a subset of signed creators on their mean performance levels, but the sample size will be limited.

[16] Note that the conventional DiD model can address both time-invariant (e.g., inherent abilities, family background) and time-variant unobservable confounders (e.g., platform popularity, economic shifts, policy changes). Time-varying confounders affecting all units equally or randomly are differenced out in the first-stage differencing (over time) in DiD. Time-invariant gaps between treatment and control groups are differenced out in the second-stage differencing (across units). We have also verified that our Deep-DiD model can address these two types of confounders through simulations, as shown in Figure 4. Time-varying confounders specific to the treated group posttreatment (e.g., positive feedback loop) cannot be differenced out but are of interest to the platform and researchers as part of the overall treatment effect.

[17] This threshold is necessary because trends are poorly defined for creators with too few videos in the pretreatment period. Although five videos are not ideal, setting a higher threshold would severely limit our sample size.

[18] Our matching procedure shares a similar spirit with the synthetic DiD method proposed by Arkhangelsky et al. (2021). Both methods aim to construct a control group with a parallel pretreatment trend relative to the treated group based on the idea that DiD requires parallel trends rather than statistical indifference in pretreatment outcomes. We choose not to use the synthetic DiD method for matching because its computationally intensive weighting and regularization parameters would have overcomplicated the implementation of our Deep-DiD method. Note that this matching procedure is not central to our Deep-DiD method; its sole purpose is to construct a control group from a very large pool of untreated units. In future applications, if the data set already has treated and control groups satisfying the parallel-trend condition, then our Deep-DiD method can be applied directly. If matching is needed, what matching method and matching variables to use depends on the specific empirical context.

[19] We have access to data from February 1, 2022, onward. For creators signed on May 1, 2022, we have 89 days of pre-signing data (February 1 to April 30).

[20] The number of "follows" a video gets means how many users decide to follow the creator after watching the video.

[21] We do not know what measures the platform uses to evaluate the impact of this signing program, nor do we have access to the exact formula the platform uses to calculate each creator's payment. Thus, we do not have a well-defined set of performance measures at hand and need to define the key performance measures by ourselves based on our understanding of the primary goals of the platform.

[22] For *User Time Contributed*, we first calculate it for each video by aggregating the total watch time a video gets in the first 20 days following the video's release, because based on historical data, usually a video gets most exposures in the first 20 days after release. Then, we aggregate it to the creator-day level by summing up the values of videos created by the creator on that day. The main reason for this handling is that the outcome variables are to capture the contribution a creator made to the platform on a day by uploading videos. If a creator has no videos uploaded on a day, then the variable value is 0 for this creator on that day. For *User Engagement*, we first follow the same procedure to aggregate each of the four interaction metrics (likes, comments, shares, and follows) from video-level to creator-day level and then conduct PCA. Factor loadings of the four components are reported in Online Appendix A.2.

[23] A caveat here is that the TWFE estimators (Columns (2) and (4) in Table 4) can be biased, as discussed in Section 1.1.

[24] Winsorization means to replace outliers with the value of specified percentiles. Winsorization can be one-sided or two-sided. For example, a 5% winsorization on the upper side (or the right tail) means to replace all values higher than the 95th percentile value with the 95th percentile value.

[25] Note that the first step of data split is by creators, whereas the second step is by creator-day level observations.

[26] A random seed is a number that initializes a pseudorandom number generator that is typically used to determine initial weights and can also affect the optimization process. Randomness in the DNN training may arise from weight initialization and mini-batch sampling in the optimization process. Training a DNN model multiple times with different random seeds and aggregating the results is a widely used practice known as *deep ensemble*, which has been shown to reduce variance, stabilize performance, and improve generalization (Lakshminarayanan et al. 2017, Fort et al. 2019, Bethard 2022). Practically, this approach is also more cost-effective than alternatives such as acquiring more data.

[27] Note that we have to constrain our comparison within the creators who were actually signed because for creators who were not signed, we cannot empirically observe their change in performance after being signed.

References

Abadie A, Gardeazabal J (2003) The economic costs of conflict: A case study of the basque country. *Amer. Econom. Rev.* 93(1):113–132.

Abadie A, Diamond A, Hainmueller J (2010) Synthetic control methods for comparative case studies: Estimating the effect of California's tobacco control program. *J. Amer. Stat. Assoc.* 105(490): 493–505.

Aggarwal CC (2018) *Neural Networks and Deep Learning*, vol. 10, No. 978 (Springer, Cham, Switzerland).

Arkhangelsky D, Athey S, Hirshberg DA, Imbens GW, Wager S (2021) Synthetic difference-in-differences. *Amer. Econom. Rev.* 111(12):4088–4118.

Ascarza E (2018) Retention futility: Targeting high-risk customers might be ineffective. *J. Marketing Res.* 55(1):80–98.

Athey S, Tibshirani J, Wager S (2019) Generalized random forests. *Ann. Statist.* 47(2):1148–1178.

Belloni A, Chernozhukov V, Fernández-Val I, Hansen C (2017) Program evaluation and causal inference with high-dimensional data. *Econometrica* 85(1):233–298.

Bengio Y (2012) Practical recommendations for gradient-based training of deep architectures. Montavon G, Orr GB, Müller K-R, eds. *Neural Networks: Tricks of the Trade*, 2nd ed. (Springer, Berlin, Heidelberg), 437–478.

Bertrand M, Duflo E, Mullainathan S (2004) How much should we trust differences-in-differences estimates? *Quart. J. Econom.* 119(1):249–275.

Bethard S (2022) We need to talk about random seeds. Preprint, submitted October 24, https://arxiv.org/abs/2210.13393.

Bhargava HK (2022) The creator economy: Managing ecosystem supply, revenue sharing, and platform design. *Management Sci.* 68(7):5233–5251.

Borusyak K, Jaravel X, Spiess J (2024) Revisiting event-study designs: Robust and efficient estimation. *Rev. Econom. Stud.* 91(6):3253–3285.

Brodie RJ, Hollebeek LD, Jurić B, Ilić A (2011) Customer engagement: Conceptual domain, fundamental propositions, and implications for research. *J. Service Res.* 14(3):252–271.

Burtch G, Hong Y, Bapna R, Griskevicius V (2018) Stimulating online reviews by combining financial incentives and social norms. *Management Sci.* 64(5):2065–2082.

Callaway B, Sant'Anna PH (2021) Difference-in-differences with multiple time periods. *J. Econom.* 225(2):200–230.

Cao J, Li X, Zhang L (2025) Is relevancy everything? A deep-learning approach to understand the effect of image-text congruence. *Management Sci.* 71(12):10579–10602.

Chaisemartin Cd, D'Haultfœuille X (2020) Two-way fixed effects estimators with heterogeneous treatment effects. *Amer. Econom. Rev.* 110(9):2964–2996.

Cheng MM, Zhang S (2024) Reputation burning: Analyzing the impact of brand sponsorship on social influencers. *Management Sci.* 71(7):5910–5932.

Chernozhukov V, Chetverikov D, Demirer M, Duflo E, Hansen C, Newey W, Robins J (2018) Double/debiased machine learning for treatment and structural parameters. *Econometrics J.* 21(1):C1–C68.

Cong Z, Liu J, Manchanda P (2021) The role of "live" in livestreaming markets: Evidence using orthogonal random forest. Preprint, submitted July 4, https://arxiv.org/abs/2107.01629.

Dew R, Ansari A, Toubia O (2022) Letting logos speak: Leveraging multiview representation learning for data-driven branding and logo design. *Marketing Sci.* 41(2):401–425.

Duflo E (2003) Grandmothers and granddaughters: Old-age pensions and intrahousehold allocation in South Africa. *World Bank Econom. Rev.* 17(1):1–25.

Dzyabura D, El Kihal S, Hauser JR, Ibragimov M (2023) Leveraging the power of images in managing product return rates. *Marketing Sci.* 42(6):1125–1142.

Farrell MH, Liang T, Misra S (2020) Deep learning for individual heterogeneity: An automatic inference framework. Preprint, submitted October 28, https://arxiv.org/abs/2010.14694.

Farrell MH, Liang T, Misra S (2021) Deep neural networks for estimation and inference. *Econometrica* 89(1):181–213.

Feng XF, Zhang S, Liu X, Srinivasan K, Lamberton CP (2025) An AI method to score celebrity visual potential. *J. Marketing Res.* 62(5):757–775

Fong NM, Fang Z, Luo X (2015) Geo-conquesting: Competitive locational targeting of mobile promotions. *J. Marketing Res.* 52(5): 726–735.

Fong H, Kumar V, Sudhir K (2022) A theory-based explainable deep learning architecture for music emotion. Preprint, submitted February 9, http://dx.doi.org/10.2139/ssrn.4025386.

Fort S, Hu H, Lakshminarayanan B (2019) Deep ensembles: A loss landscape perspective. Preprint, submitted December 5, https://arxiv.org/abs/1912.02757.

Goodfellow I, Bengio Y, Courville A (2016) *Deep Learning* (MIT Press, Cambridge, MA).

Guo T, Sriram S, Manchanda P (2021) The effect of information disclosure on industry payments to physicians. *J. Marketing Res.* 58(1):115–140.

Hill JL (2011) Bayesian nonparametric modeling for causal inference. *J. Comput. Graph. Statist.* 20(1):217–240.

Hitsch GJ, Misra S, Zhang WW (2024) Heterogeneous treatment effects and optimal targeting policy evaluation. *Quant. Marketing Econom.* 22(2):115–168.

Huang Y, Morozov I (2025) The promotional effects of live streams by twitch influencers. *Marketing Sci.* 44(4):916–932.

Huang N, Burtch G, Gu B, Hong Y, Liang C, Wang K, Fu D, Yang B (2019) Motivating user-generated content with performance feedback: Evidence from randomized field experiments. *Management Sci.* 65(1):327–345.

Hwang S, Liu X, Srinivasan K (2021) Voice analytics of online influencers. Preprint, submitted February 18, http://dx.doi.org/10.2139/ssrn.3773825.

Jain S, Qian K (2021) Compensating online content producers: A theoretical analysis. *Management Sci.* 67(11):7075–7090.

Lakshminarayanan B, Pritzel A, Blundell C (2017) Simple and scalable predictive uncertainty estimation using deep ensembles. Guyon I, Luxburg UV, Bengio S, Wallach H, Fergus R,

Vishwanathan S, Garnett R, eds. *Proc. 31st Internat. Conf. Neural Inform. Processing Systems* (Curran Associates, Inc., Red Hook, NY), 6405–6416.

Leung FF, Gu FF, Li Y, Zhang JZ, Palmatier RW (2022) Influencer marketing effectiveness. *J. Marketing* 86(6):93–115.

Li Y, Xie Y (2020) Is a picture worth a thousand words? An empirical study of image content and social media engagement. *J. Marketing Res.* 57(1):1–19.

Li KT, Luo L, Pattabhiramaiah A (2024) Causal inference with quasi-experimental data. *IMPACT at JMR*. https://www.ama.org/marketingnews/causal-inference-with-quasi-experimental-data.

Li X, Shi M, Wang XS (2019) Video mining: Measuring visual information using automatic methods. *Internat. J. Res. Marketing* 36(2):216–231.

Liu X (2023) Deep learning in marketing: A review and research agenda. *Artificial Intelligence Marketing* (Emerald Publishing Limited, Bingley, UK), 239–271.

Liu L, Dzyabura D, Mizik N (2020) Visual listening in: Extracting brand image portrayed on social media. *Marketing Sci.* 39(4): 669–686.

Liu X, Lee D, Srinivasan K (2019) Large-scale cross-category analysis of consumer review content on sales conversion leveraging deep learning. *J. Marketing Res.* 56(6):918–943.

Liu Z, Zhang W, Liu X, Muller E, Xiong F (2022) Success and survival in livestream shopping. Preprint, submitted February 9, http://dx.doi.org/10.2139/ssrn.4028092.

Lu S, Xie Y, Chen X (2023) Immediate and enduring effects of digital badges on online content consumption and generation. *Internat. J. Res. Marketing* 40(1):146–163.

Lu S, Yao D, Chen X, Grewal R (2021) Do larger audiences generate greater revenues under pay what you want? Evidence from a live streaming platform. *Marketing Sci.* 40(5):964–984.

Oestreicher-Singer G, Zalmanson L (2013) Content or community? A digital business strategy for content providers in the social age. *MIS Quart.* 37(2):591–616.

Oprescu M, Syrgkanis V, Wu ZS (2019) Orthogonal random forest for causal inference. *Internat. Conf. Machine Learn.* (PMLR), 4932–4941.

Rajaram P, Manchanda P (2021) Unboxing engagement in YouTube influencer videos: An attention-based approach. Preprint, submitted January 25, http://dx.doi.org/10.2139/ssrn.3752107.

Roth J, Sant'Anna PH, Bilinski A, Poe J (2023) What's trending in difference-in-differences? A synthesis of the recent econometrics literature. *J. Econom.* 235(2):2218–2244.

Simester D, Timoshenko A, Zoumpoulis SI (2020) Targeting prospective customers: Robustness of machine-learning methods to typical data challenges. *Management Sci.* 66(6):2495–2522.

Sun L, Abraham S (2021) Estimating dynamic treatment effects in event studies with heterogeneous treatment effects. *J. Econom.* 225(2):175–199.

Tian Z, Dew R, Iyengar R (2024) Mega or micro? Influencer selection using follower elasticity. *J. Marketing Res.* 61(3):472–495.

Wager S, Athey S (2018) Estimation and inference of heterogeneous treatment effects using random forests. *J. Amer. Statist. Assoc.* 113(523):1228–1242.

Wang X, Lu S, Li X, Khamitov M, Bendle N (2021) Audio mining: The role of vocal tone in persuasion. *J. Consumer Res.* 48(2):189–211.

Wei Y, Jiang Z (2022) Estimating parameters of structural models using neural networks. USC Marshall School of Business Research Paper.

Wies S, Bleier A, Edeling A (2023) Finding goldilocks influencers: How follower count drives social media engagement. *J. Marketing* 87(3):383–405.

Yang M, Ren Y, Adomavicius G (2019) Understanding user-generated content and customer engagement on facebook business pages. *Inform. Systems Res.* 30(3):839–855.

Yang J, Zhang J, Zhang Y (2025) Engagement that sells: Influencer video advertising on TikTok. *Marketing Sci.* 44(2):247–267.

Zeng Z, Dai H, Zhang DJ, Zhang H, Zhang R, Xu Z, Shen Z-JM (2022) The impact of social nudges on user-generated content for social network platforms. *Management Sci.* 69(9):5189–5208.

Zhang WW (2024) Optimal comprehensible targeting. PhD thesis, University of Chicago Booth School of Business, Chicago.

Zhang M, Luo L (2023) Can consumer-posted photos serve as a leading indicator of restaurant survival? Evidence from Yelp. *Management Sci.* 69(1):25–50.

Zhang S, Lee D, Singh PV, Srinivasan K (2022) What makes a good image? Airbnb demand analytics leveraging interpretable image features. *Management Sci.* 68(8):5644–5666.

Zhou M, Chen GH, Ferreira P, Smith MD (2021) Consumer behavior in the online classroom: Using video analytics and machine learning to understand the consumption of video courseware. *J. Marketing Res.* 58(6):1079–1100.

Copyright of Marketing Science (INFORMS) is the property of INFORMS: Institute for Operations Research & the Management Sciences and its content may not be copied or emailed to multiple sites without the copyright holder's express written permission. Additionally, content may not be used with any artificial intelligence tools or machine learning technologies. However, users may print, download, or email articles for individual use.